

Text2Image

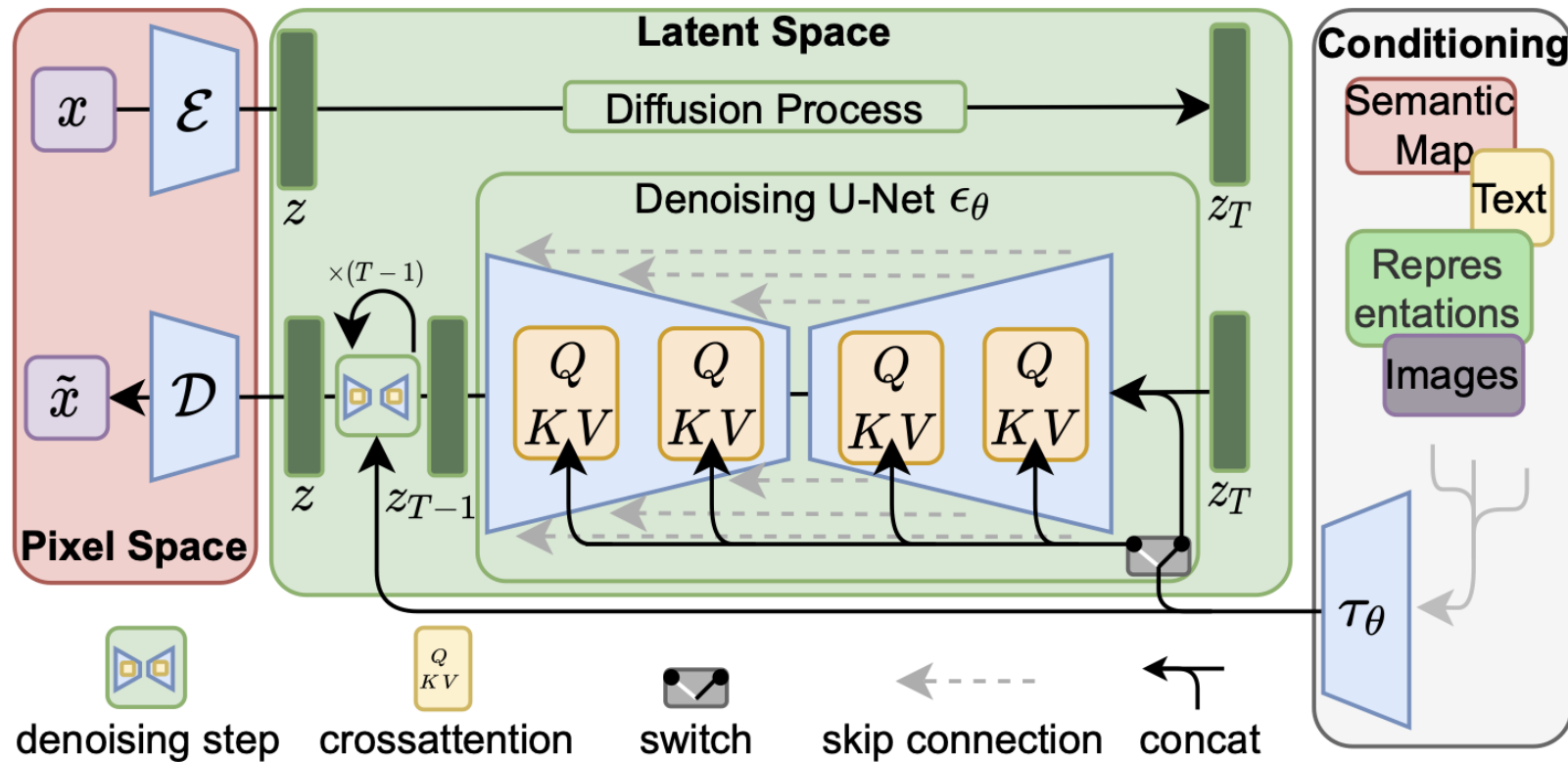
Amit Shmidov | Tutorial 9

A play in 6 acts

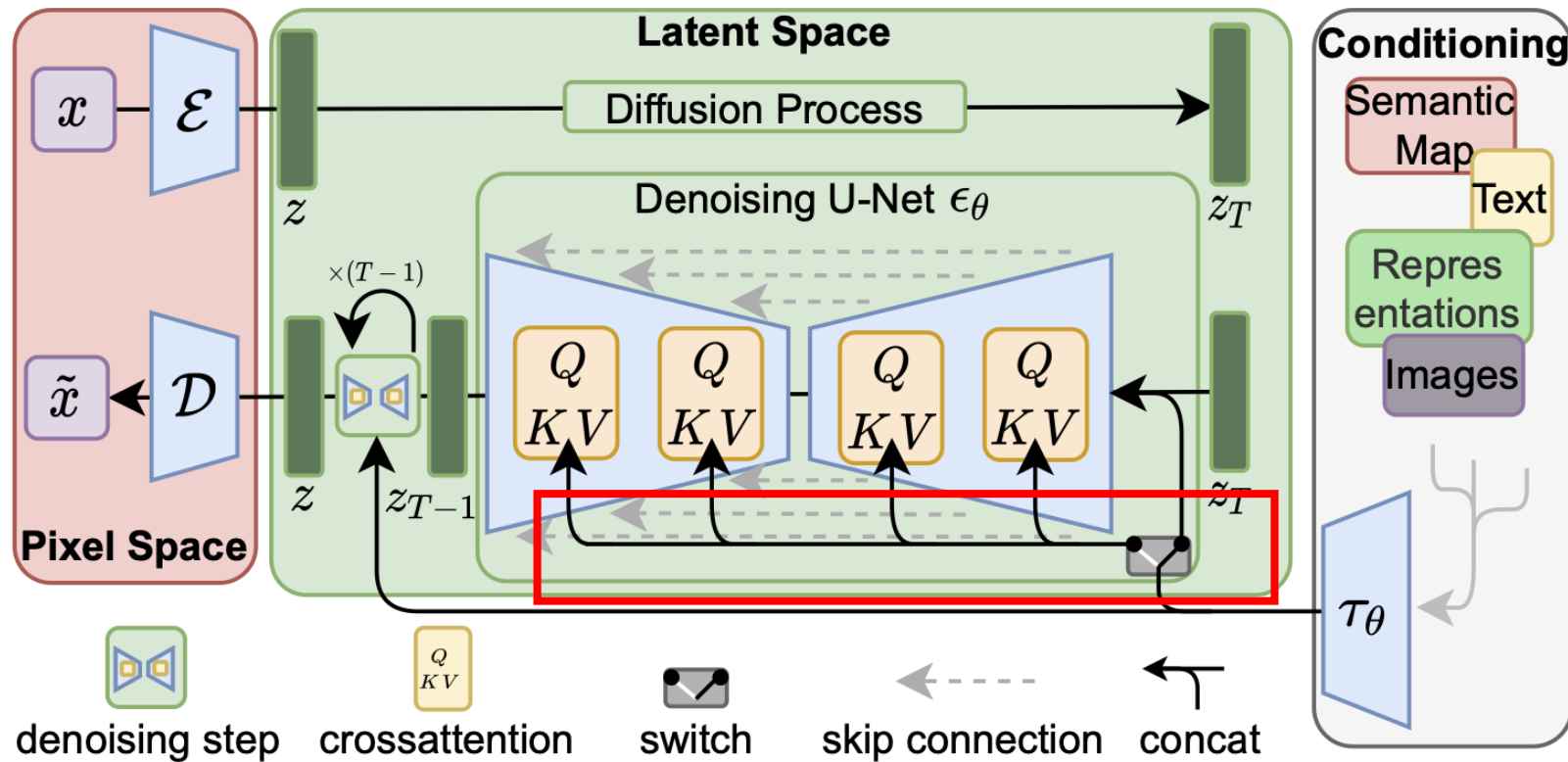
1. Cross attention
2. CLIP
3. Text encoders
4. Latent Spaces
5. Classifier-free guidance
6. DiT

1 | Cross Attention

How does text enter a diffusion model?



How does text enter a diffusion model?



Cross attention

Given a prompt y and an embedding method $\tau_\theta(y) \in \mathbb{R}^{M \times d_\tau}$:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) \cdot V$$

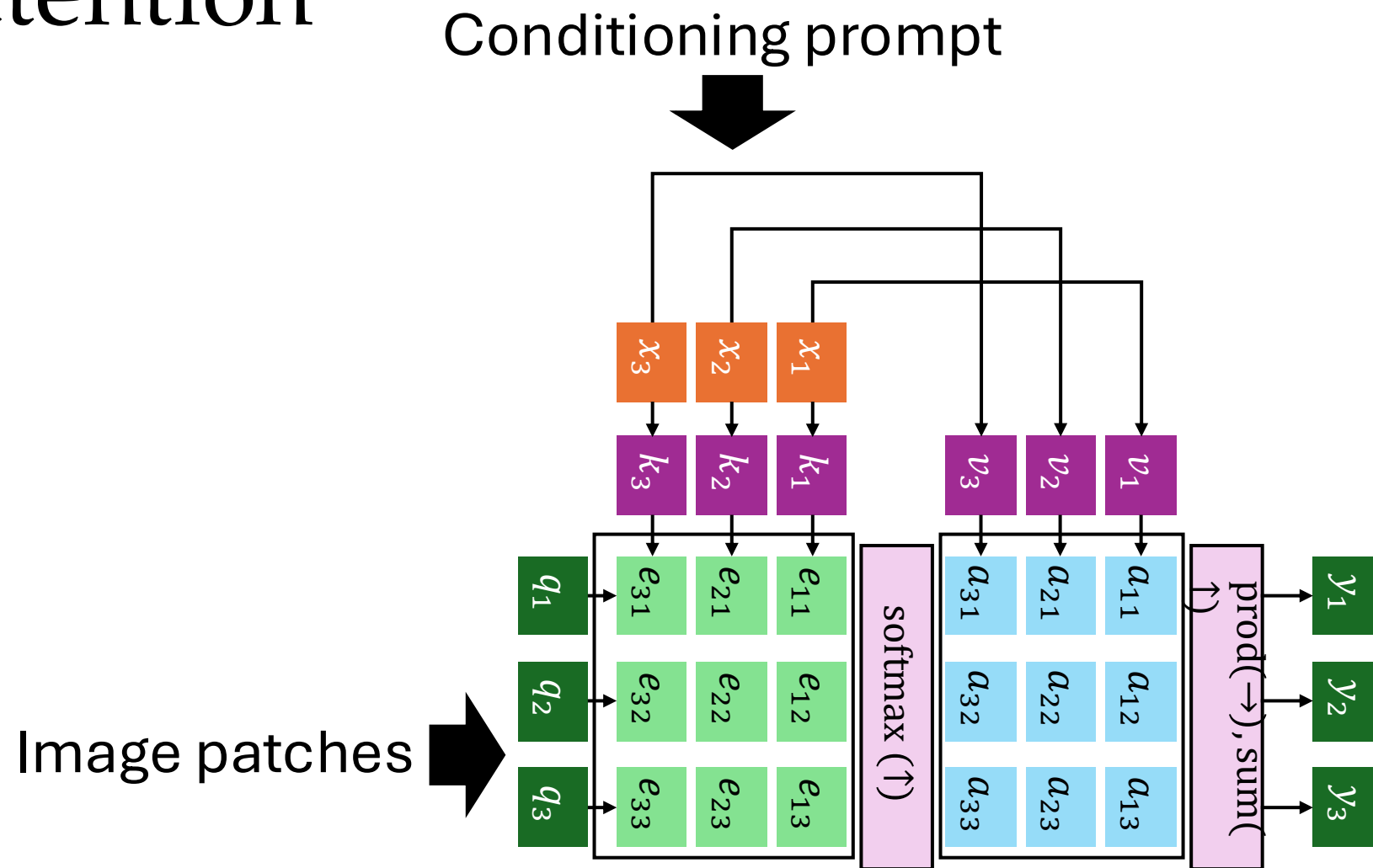
$$Q = W_Q^{(i)} \cdot \varphi_i(z_t)$$

$$K = W_K^{(i)} \cdot \tau_\theta(y)$$

$$V = W_V^{(i)} \cdot \tau_\theta(y)$$

Where $\varphi(z_t)$ is an intermediate representation and W are learnable projection matrices.

Cross attention



2 | CLIP

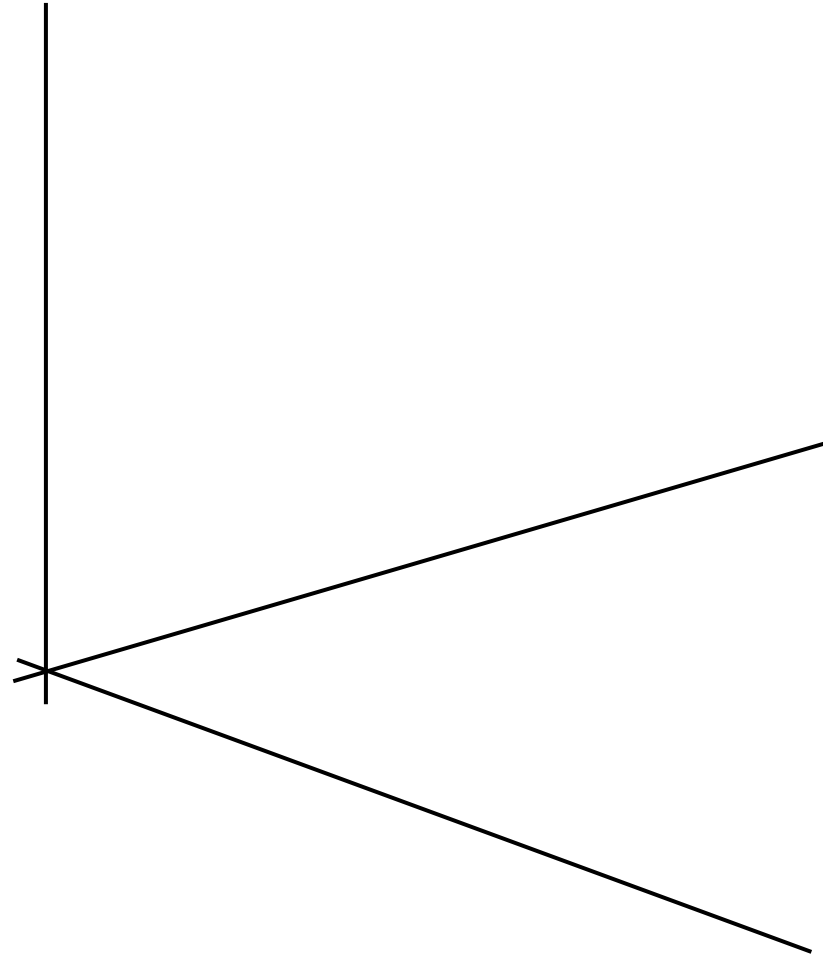
Shared embedding space for image and text



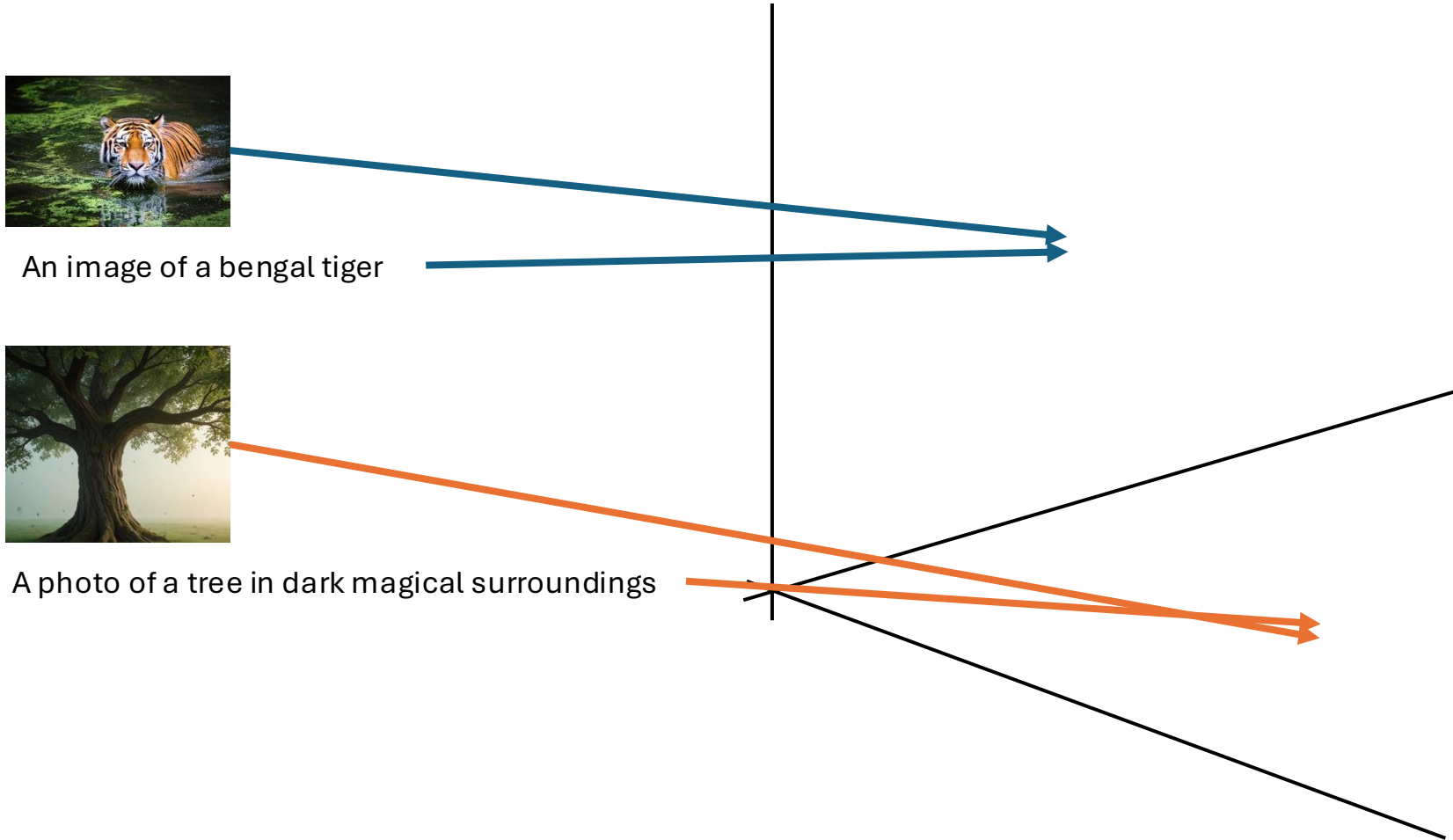
An image of a bengal tiger



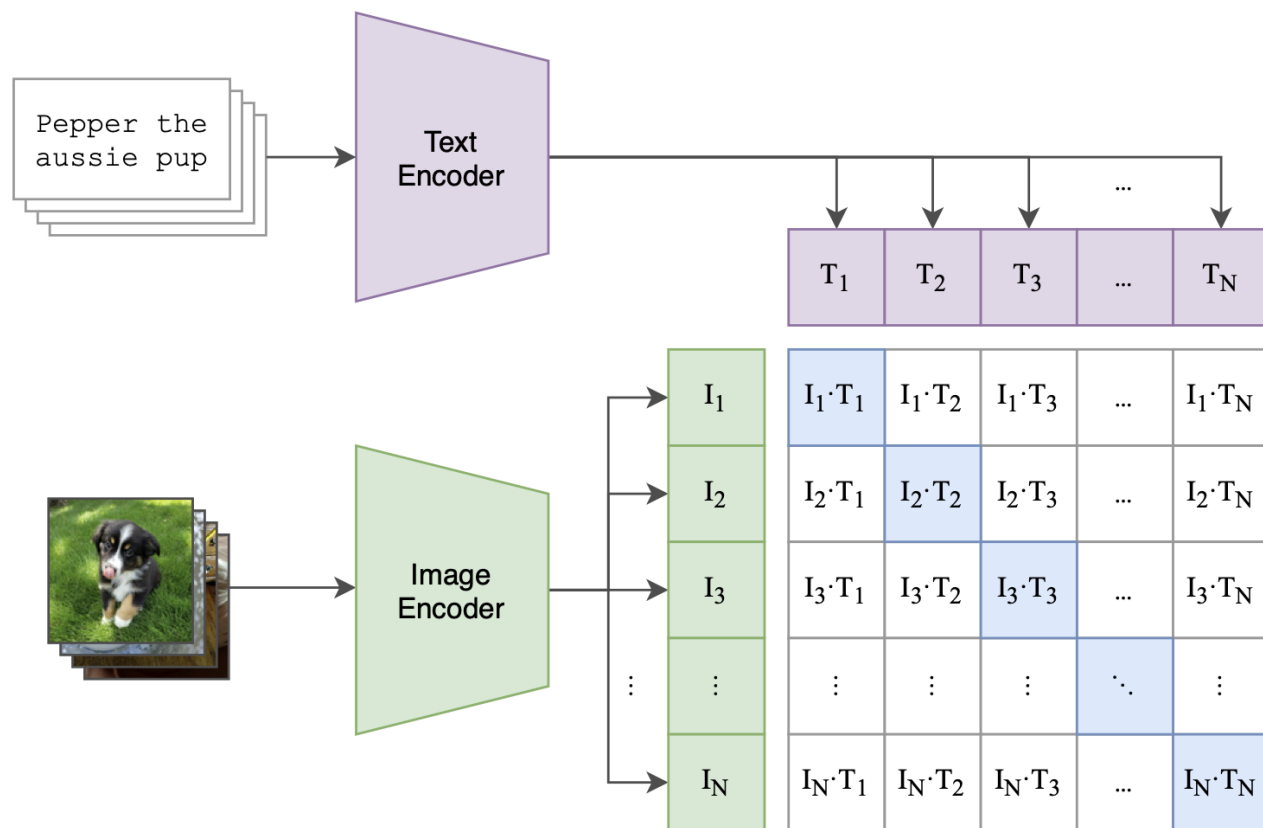
A photo of a tree in dark magical surroundings



Shared embedding space for image and text



Contrastive Image-Language Pretraining



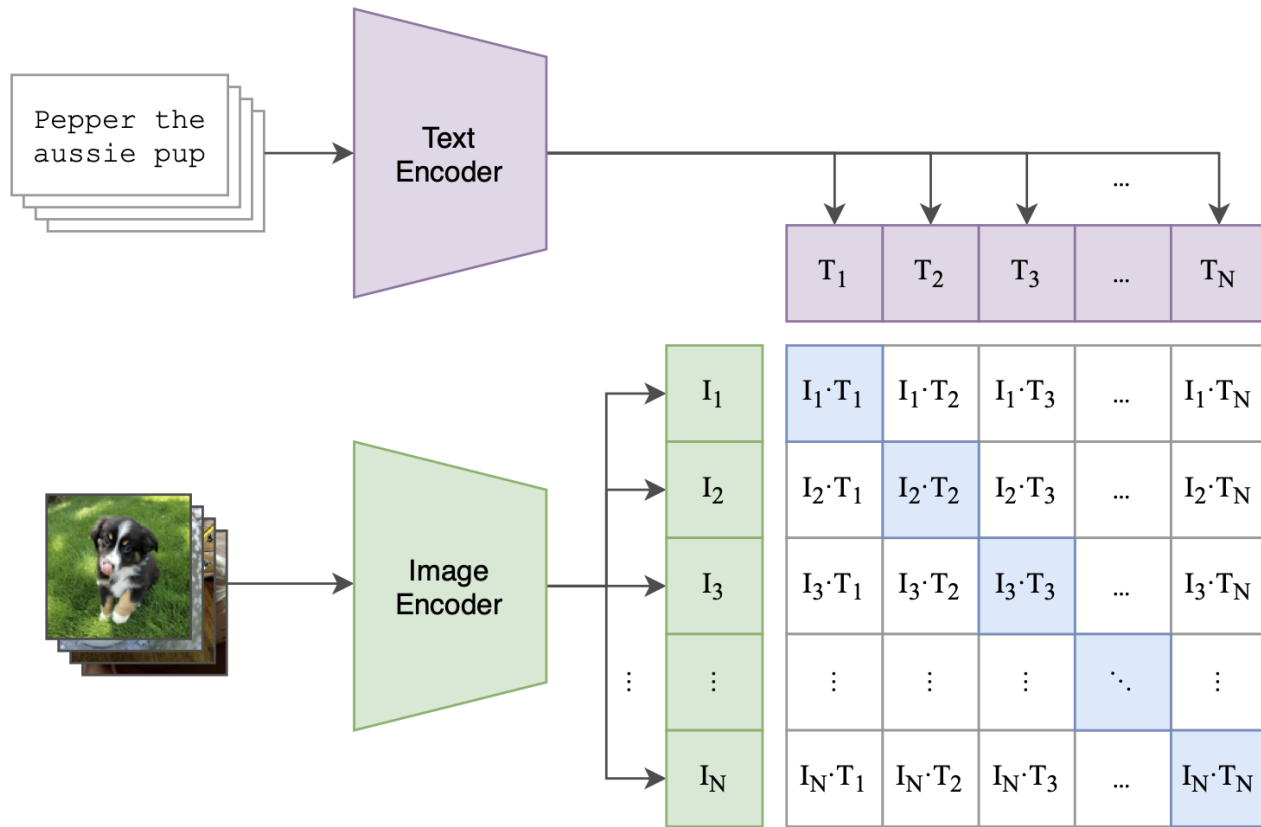
Trained on 400 million image-text pairs.

Transformer as text encoder.

ResNet or ViT as image encoder.

Contrastive loss.

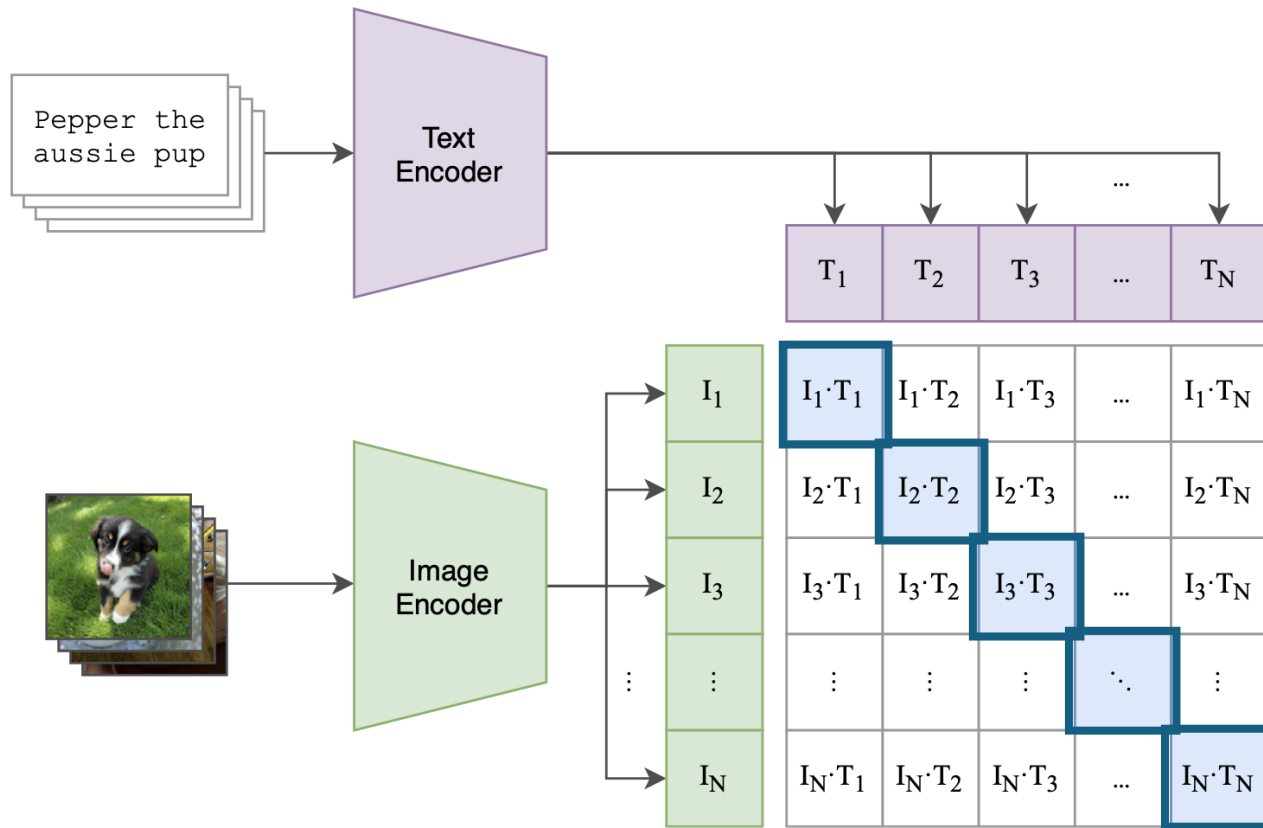
Contrastive loss



Positive pairs:

Negative pairs:

Contrastive loss

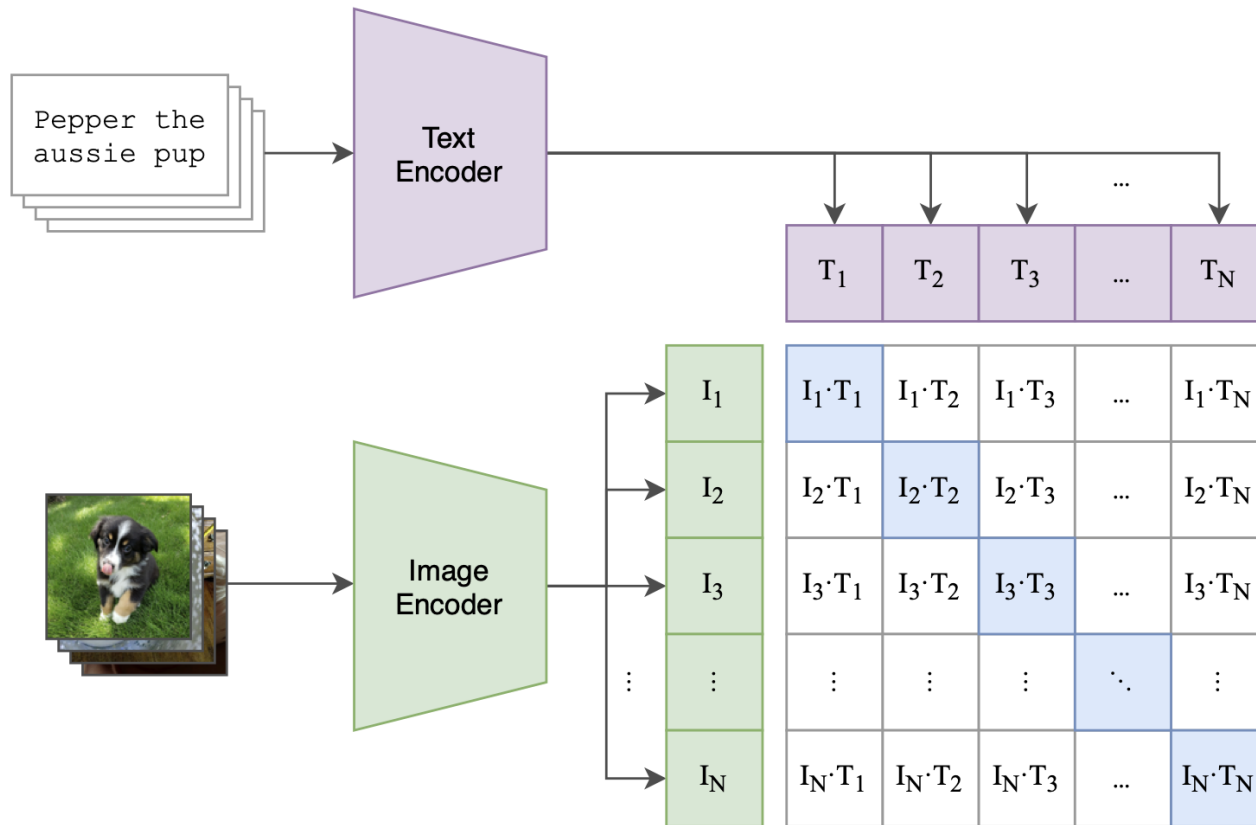


Positive pairs:

Aligned text-image pairs ($N \times 1$).

Negative pairs:

Contrastive loss

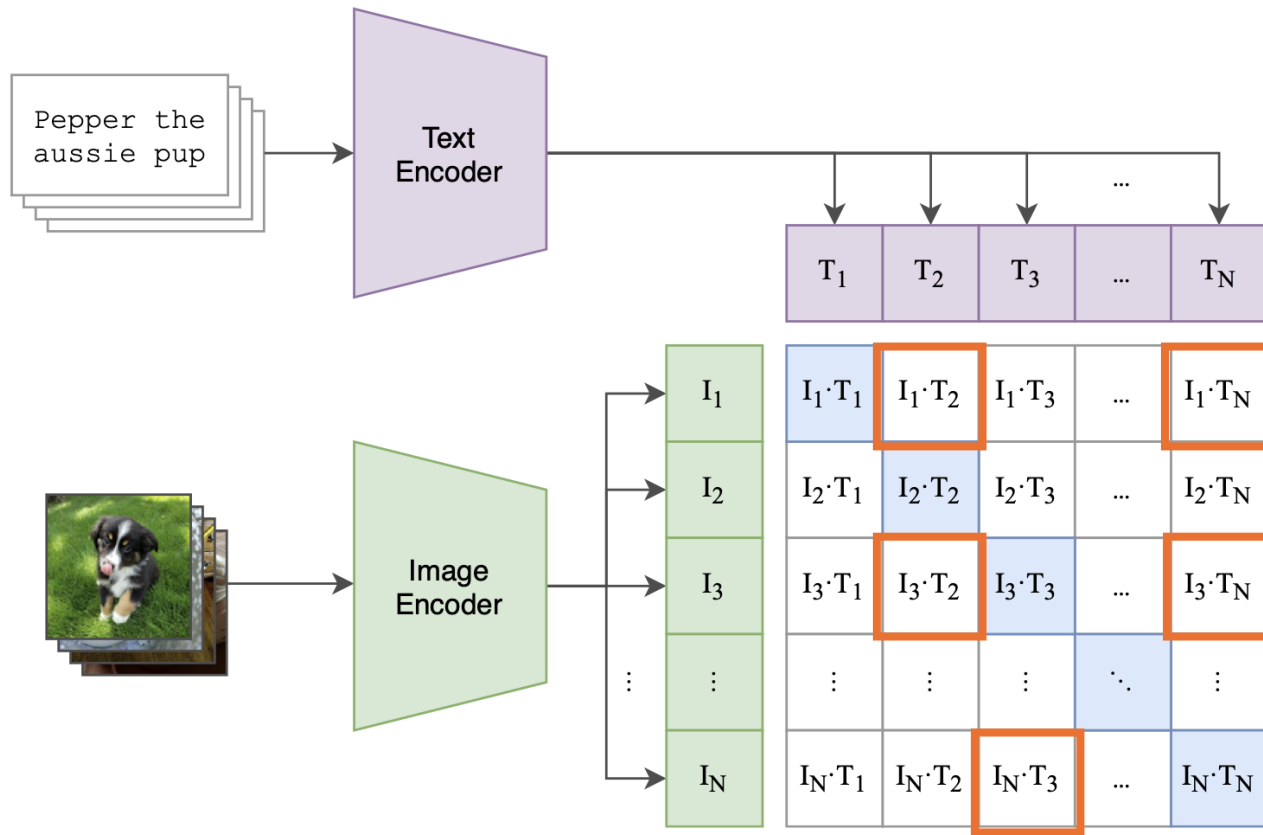


Positive pairs:

Aligned text-image pairs ($N \times 1$).

Negative pairs:

Contrastive loss



Positive pairs:

Aligned text-image pairs (N).

Negative pairs:

Misaligned text-image pairs ($N^2 - N$).

Contrastive loss



x_1

An image of a bengal tiger

y_1



x_2

A photo of a tree in dark magical surroundings

y_2

$$L_{i2t} = -\frac{1}{N} \sum_i \log \frac{\exp\left(\frac{x_i^\top y_i}{\tau}\right)}{\sum_{j=1}^N \exp\left(\frac{x_i^\top y_j}{\tau}\right)}$$

$$L_{t2i} = -\frac{1}{N} \sum_i \log \frac{\exp\left(\frac{y_i^\top x_i}{\tau}\right)}{\sum_{j=1}^N \exp\left(\frac{y_i^\top x_j}{\tau}\right)}$$

$$L = \frac{1}{2} (L_{i2t} + L_{t2i})$$

Pseudo code implementation

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t            - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

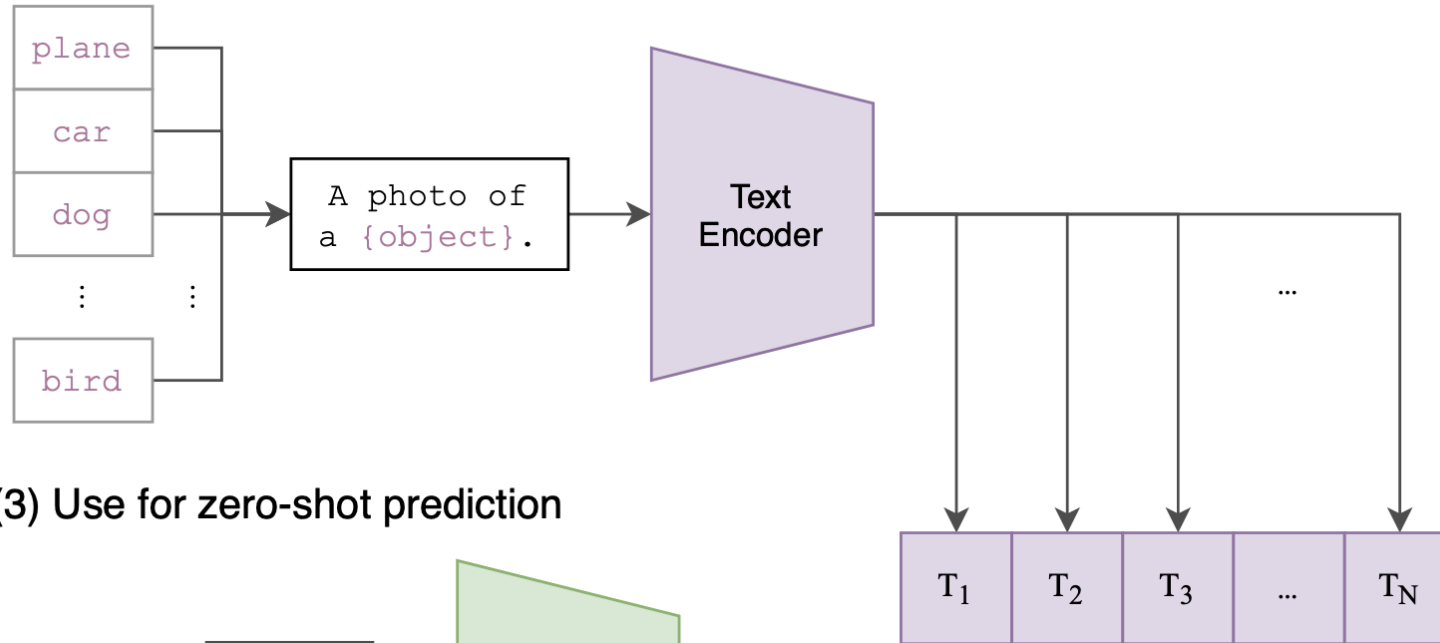
# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

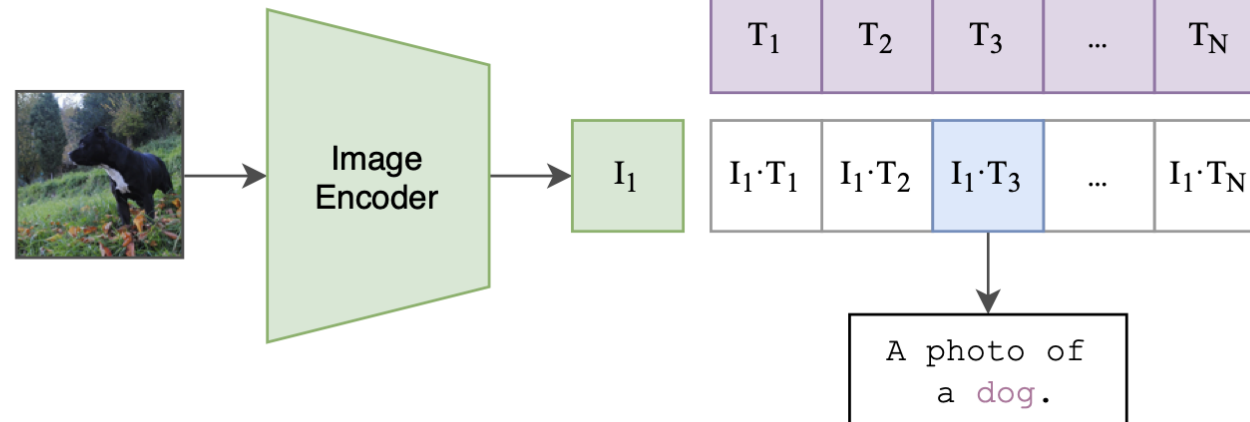
# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

Zero-shot prediction

(2) Create dataset classifier from label text



(3) Use for zero-shot prediction



CLIP as a text encoder for T2I

CLIP final output is a single global embedding.

Can it be used for T2I?

CLIP as a text encoder for T2I

CLIP final output is a single global embedding.

Can it be used for T2I? **No, it is too coarse-grained.**

CLIP as a text encoder for T2I

CLIP final output is a single global embedding.

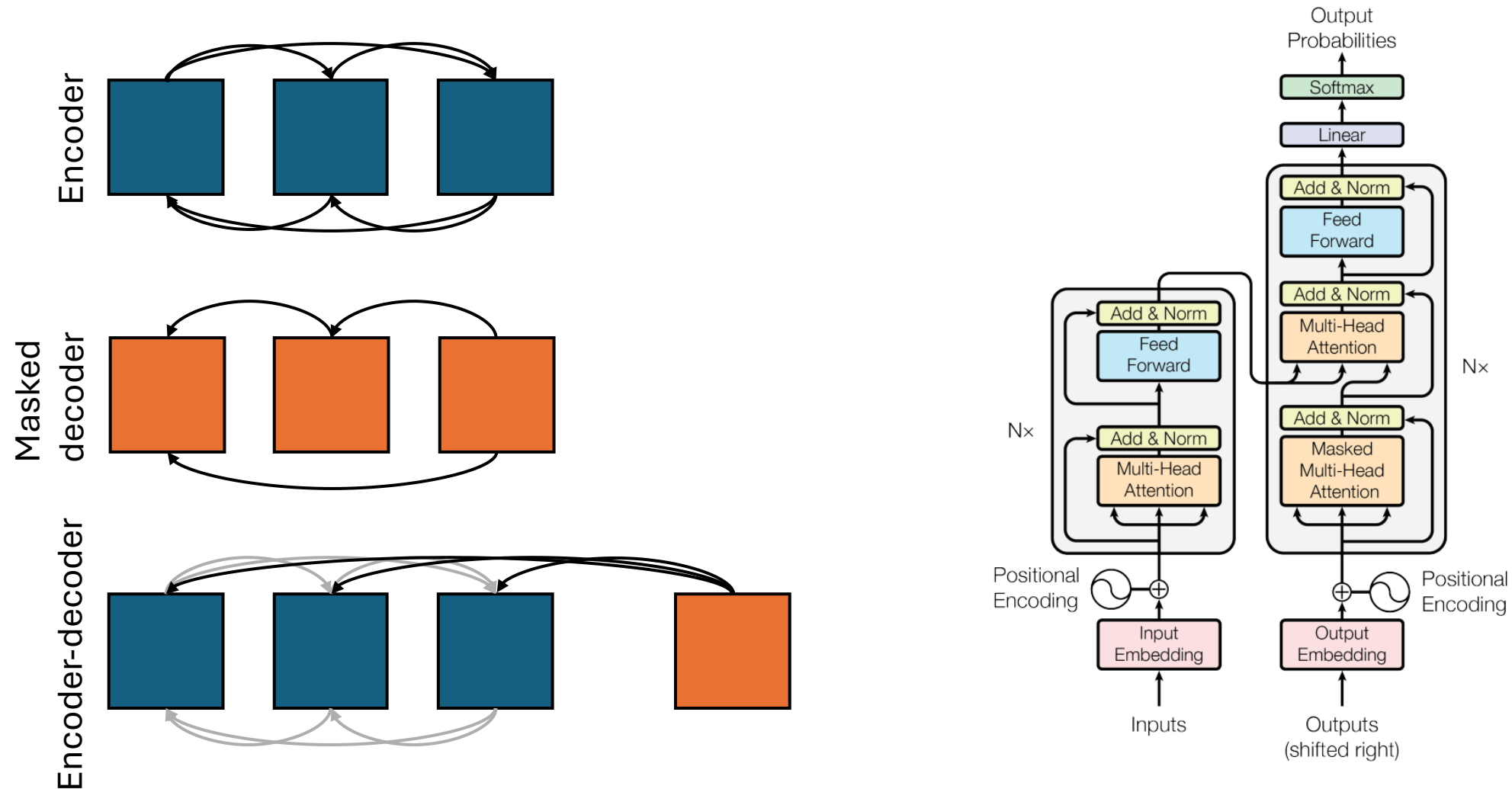
Can it be used for T2I? **No, it is too coarse-grained.**

CLIP internally generates per-token embeddings, which it then pools for a global text-image alignment.

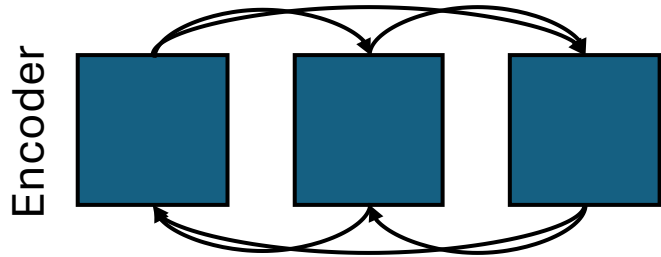
Stable diffusion 1 and 2 use these per-token encodings.

3 | Text Encoders

Types of transformer architectures in LM

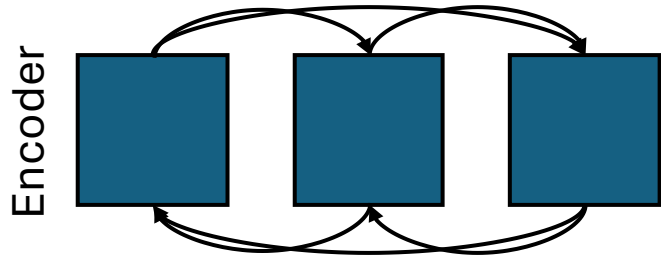


Types of transformer architectures in LM

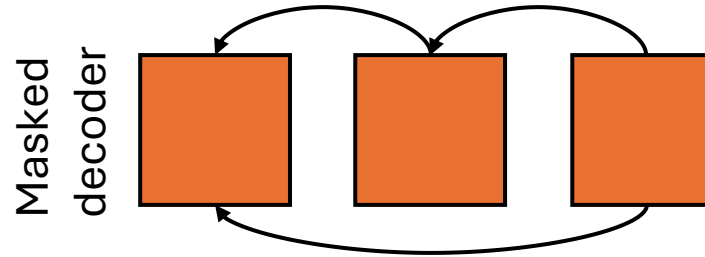


Examples: BERT, RoBERTa
Bi-directional text inferring

Types of transformer architectures in LM

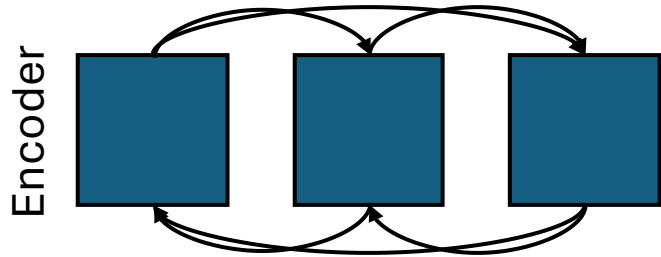


Examples: BERT, RoBERTa
Bi-directional text inferring

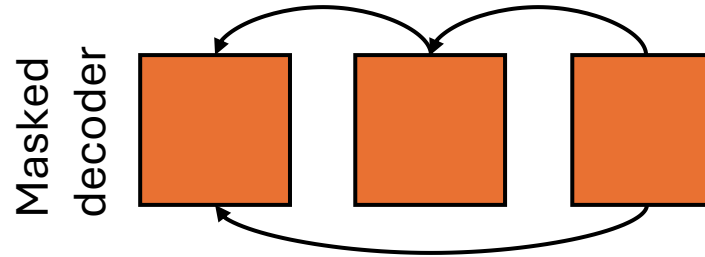


Examples: GPT-2, GPT-3
Causal text generation

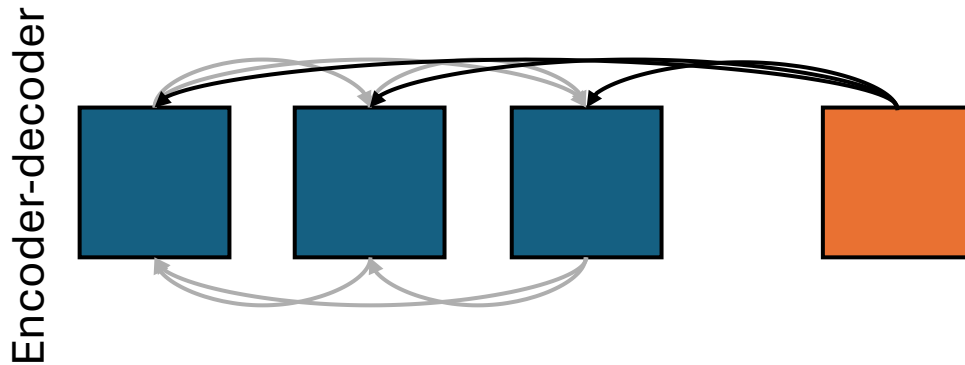
Types of transformer architectures in LM



Examples: BERT, RoBERTa
Bi-directional text inferring

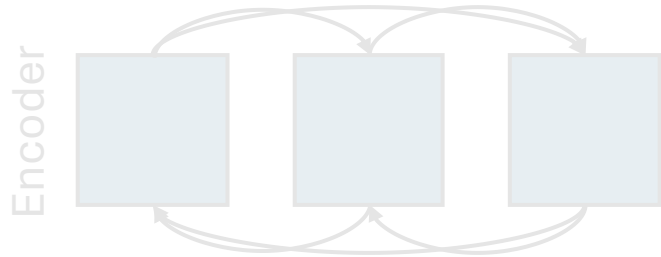


Examples: GPT-2, GPT-3
Causal text generation

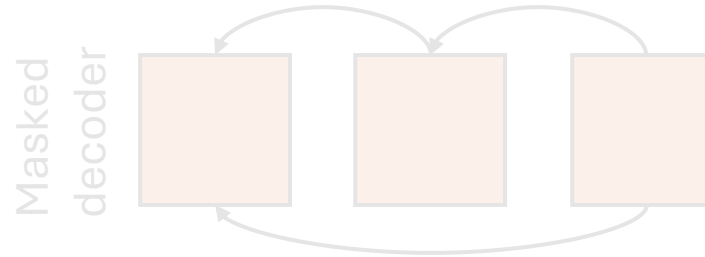


Examples: T5
Text generation conditionally on encoded input

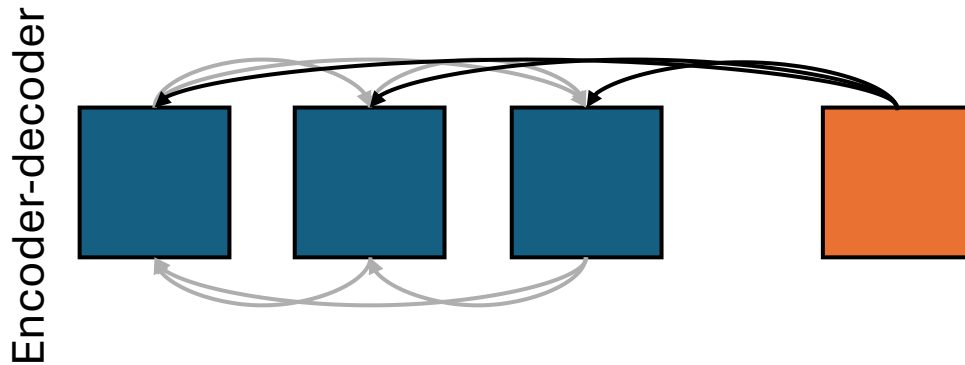
Types of transformer architectures in LM



Examples: BERT, RoBERTa
Bi-directional text inferring



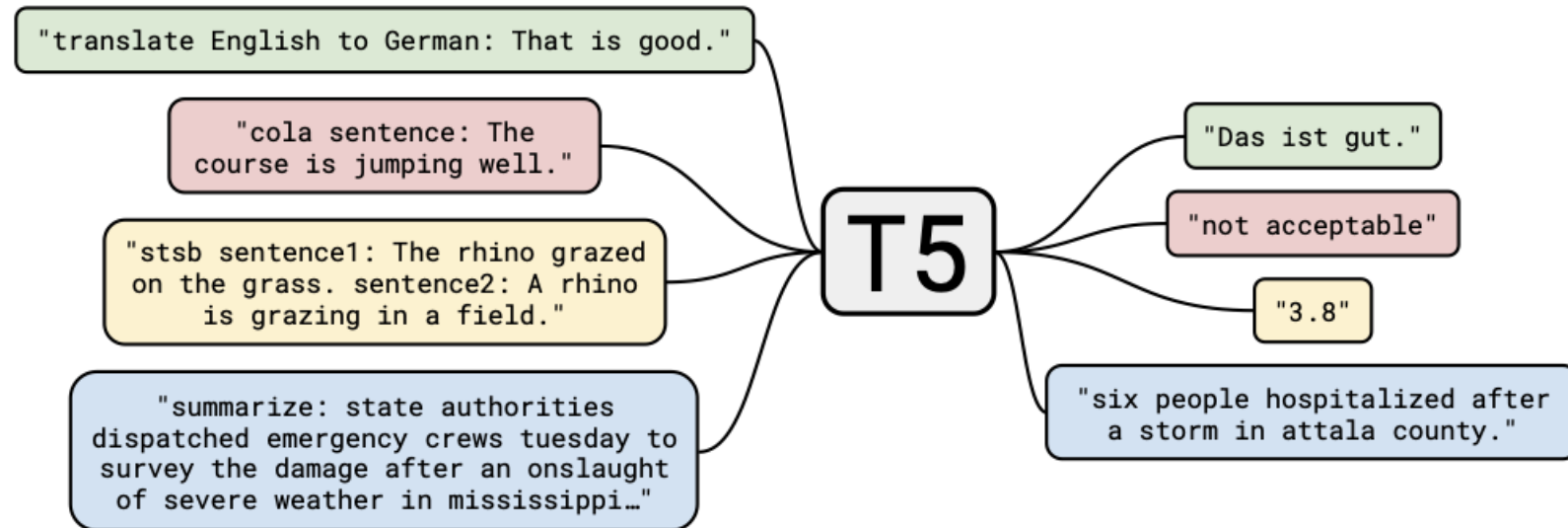
Examples: GPT-2, GPT-3
Causal text generation



Examples: T5
Text generation conditionally on encoded input

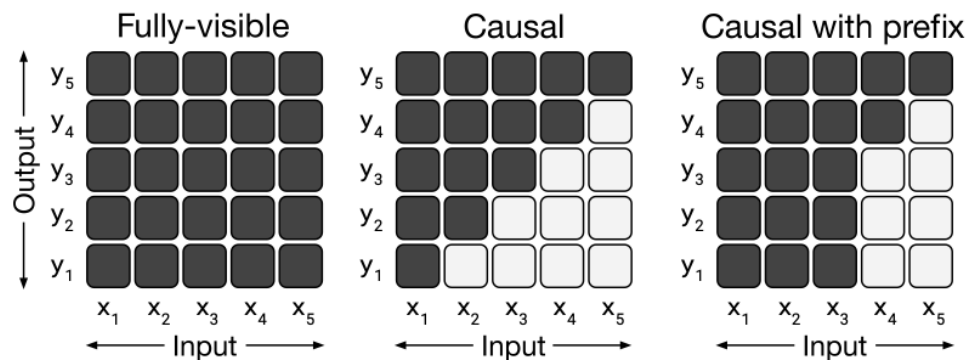
Today

T5: Text-to-Text Transfer Transformer



T5: Text-to-Text Transfer Transformer

Model architectures:



Pretraining objectives:

Objective	Inputs	Targets
Prefix language modeling	Thank you for inviting	me to your party last week .
BERT-style Devlin et al. (2018)	Thank you <M> <M> me to your party apple week .	(original text)
Deshuffling	party me for your to . last fun you inviting week Thank	(original text)
MASS-style Song et al. (2019)	Thank you <M> <M> me to your party <M> week .	(original text)
I.i.d. noise, replace spans	Thank you <X> me to your party <Y> week .	<X> for inviting <Y> last <Z>
I.i.d. noise, drop tokens	Thank you me to your party week .	for inviting last
Random spans	Thank you <X> to <Y> week .	<X> for inviting me <Y> your party last <Z>

Architecture	Objective	Params	Cost	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ Encoder-decoder	Denoising	$2P$	M	83.28	19.24	80.88	71.36	26.98	39.82	27.65
Enc-dec, shared	Denoising	P	M	82.81	18.78	80.63	70.73	26.72	39.03	27.46
Enc-dec, 6 layers	Denoising	P	$M/2$	80.88	18.97	77.59	68.42	26.38	38.40	26.95
Language model	Denoising	P	M	74.70	17.93	61.14	55.02	25.09	35.28	25.86
Prefix LM	Denoising	P	M	81.82	18.61	78.94	68.11	26.43	37.98	27.39
Encoder-decoder	LM	$2P$	M	79.56	18.59	76.02	64.29	26.27	39.17	26.86
Enc-dec, shared	LM	P	M	79.60	18.13	76.35	63.50	26.62	39.17	27.05
Enc-dec, 6 layers	LM	P	$M/2$	78.67	18.26	75.32	64.06	26.13	38.42	26.89
Language model	LM	P	M	73.78	17.54	53.81	56.51	25.23	34.31	25.38
Prefix LM	LM	P	M	79.68	17.84	76.87	64.86	26.28	37.51	26.76

Objective	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
BERT-style (Devlin et al., 2018)	82.96	19.17	80.65	69.85	26.78	40.03	27.41
MASS-style (Song et al., 2019)	82.32	19.16	80.10	69.28	26.79	39.89	27.55
★ Replace corrupted spans	83.28	19.24	80.88	71.36	26.98	39.82	27.65
Drop corrupted tokens	84.44	19.31	80.52	68.67	27.07	39.76	27.82

Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. Raffel et al., 2019.

T5 text as encoder for T2I

The encoder is used to encode the text and the decoder is discarded.
No need for task-specific prompt template.

Encoding pipeline:

Prompt → Tokenizer → Encoder → **Hidden state**

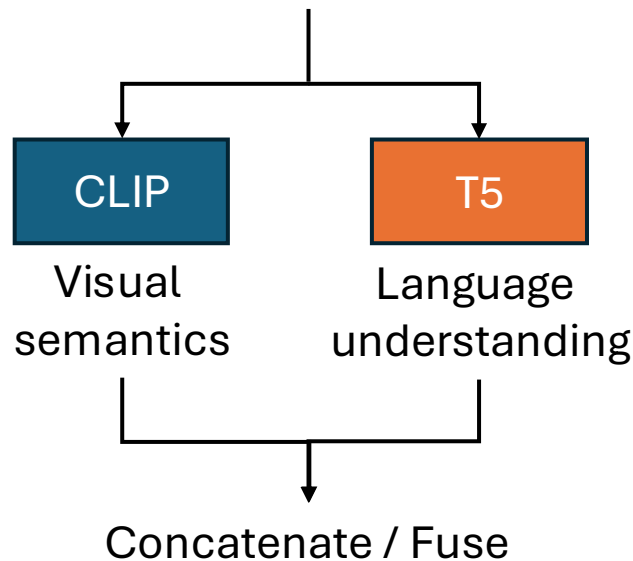
Why CLIP is not enough?

When T2I prompts become more complex, CLIP's “language modeling” is not enough:

- CLIP learns $f(T) \approx f(I)$ while T5 learns $P(T)$.
- T5 learns a rich multi-task contextual representation, that can be leveraged for T2I.
- Prompt following requires text understanding, not simple image-text alignment.

CLIP + T5

Many modern models (FLUX, SD3) use both:



CLIP: object concepts, image-language alignment

T5: grammar, long prompts, reasoning, instruction following

VLM as text encoder for T2I

Modern VLMs are capable of complex reasoning, understanding and text generation with very large context.

VLM as text encoder for T2I

Modern VLMs are capable of complex reasoning, understanding and text generation with very large context.

VLMs do not only encode the prompt, they **refine and expand** it.

VLM as text encoder for T2I

Modern VLMs are capable of complex reasoning, understanding and text generation with very large context.

VLMs do not only encode the prompt, they **refine and expand** it.

Different strategies for generating embeddings:

- Final layer hidden states
- Layer intermediate hidden states
- Hidden state projection

4 | Latent Spaces

Latent spaces

We saw how actions in latent spaces can help us in segmentation.
Why are they important for image generation or editing?

Latent spaces

We saw how actions in latent spaces can help us in segmentation.
Why are they important for image generation or editing?

Images occupy a very high-dimensional space!

3 color channels, 600 px height and 1000 px width define a unique vector of **1.8 million** elements!

Latent spaces

Images occupy a very high-dimensional space!

3 color channels, 600 px height and 1000 px width define a unique vector of **1.8 million** elements!

Core issues:

- Memory blow up
- Learning problem is hard
- Pixel space is semantically and visually redundant

Latent spaces

Why is this a problem in diffusion models and not in supervised learning?

Images occupy a very high-dimensional space!
3 color channels, 600 px height and 1000 px width define a unique vector of **1.8 million** elements!

Core issues:

- Memory blow up
- Learning problem is hard
- Pixel space is semantically and visually redundant

Latent spaces

Why is this a problem in diffusion models and not in supervised learning?

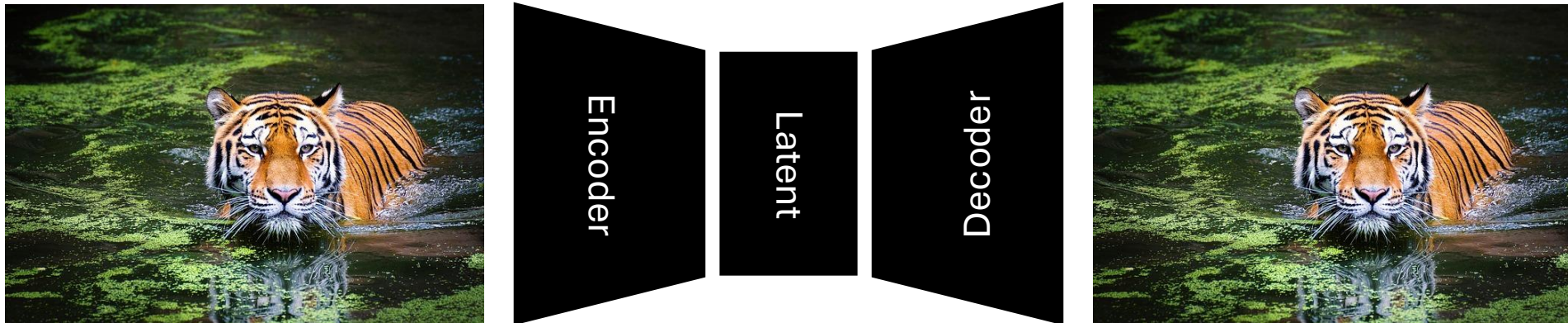
We learn a vector field $u_t^\theta: \mathbb{R}^d \rightarrow \mathbb{R}^d$ and apply it many times (simulation of ODE).

Core issues:

- Memory blow up
- Learning problem is hard
- Pixel space is semantically and visually redundant

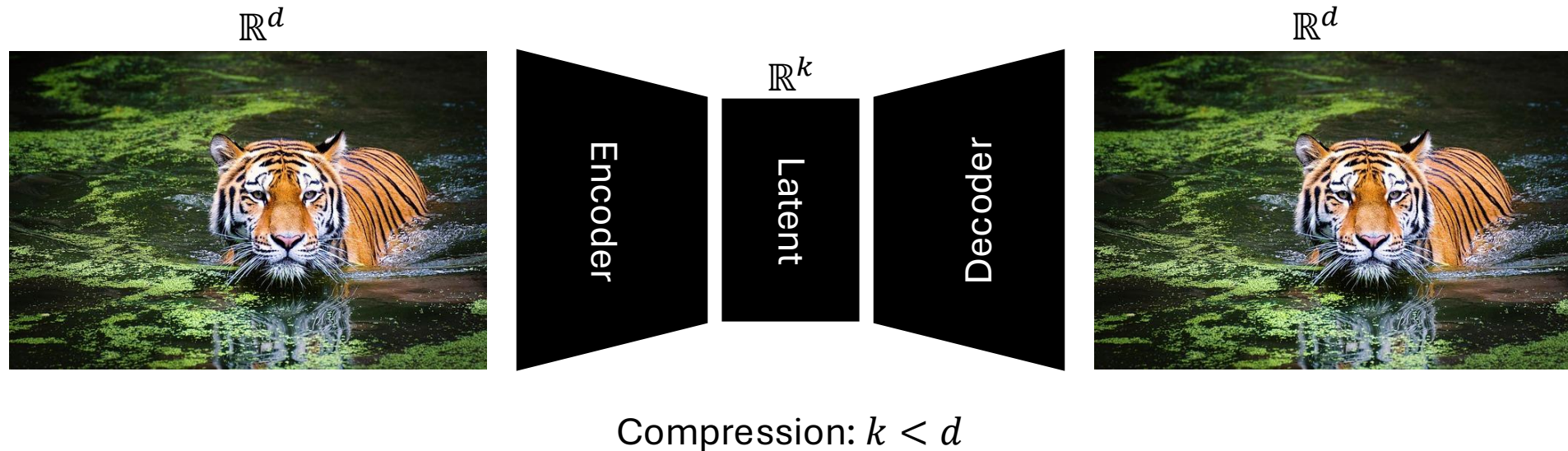
Autoencoders

Latent spaces are constructed via autoencoders



Autoencoders

Latent spaces are constructed via autoencoders



Autoencoders

What happens to the data distribution when we transform it into latent space?

Autoencoders

What happens to the data distribution when we transform it into latent space?

The simple answer: we don't know, the reconstruction loss doesn't tell anything about the distribution.

The problem: We might make the learning problem even harder – we can compress but not be able to learn how to sample from the distribution anymore.

Autoencoders

What happens to the data distribution when we transform it into latent space?

The simple answer: we don't know, the reconstruction loss doesn't tell anything about the distribution.

The problem: We might make the learning problem even harder – we can compress but not be able to learn how to sample from the distribution anymore.

Goal: use autoencoders to create “nice” latent spaces.

KL-divergence

Given two probability densities p and q , the Kullback-Leibler divergence (KL -divergence) is:

$$D_{KL}(q(x)||p(x)) = \int q(x) \log \frac{q(x)}{p(x)} dx = E_{X \sim q} \left[\log \frac{q(X)}{p(X)} \right]$$

It measures how two distribution are different from each other:

$$\begin{aligned} D_{KL}(q(x)||p(x)) &\geq 0 \\ D_{KL}(q(x)||p(x)) &= 0 \leftrightarrow q = p \end{aligned}$$

KL-divergence of normal distributions

Given two normal distributions:

$$q(x) = \mathcal{N}(x; \mu_q, \sigma_q^2 I_d), \quad p(x) = \mathcal{N}(x; \mu_p, \sigma_p^2 I_d)$$

The KL -divergence is given by:

$$D_{KL}(q||p) = \frac{1}{2} \left(\mathcal{K} \left(\frac{\sigma_q^2}{\sigma_p^2} \right) + \frac{||\mu_q - \mu_p||^2}{\sigma_p^2} \right)$$
$$\mathcal{K} = \sum_{i=1}^d (\alpha_i - \log \alpha_i - 1)$$

KL-divergence of normal distributions

Given two normal distributions:

$$q(x) = \mathcal{N}(x; \mu_q, \sigma_q^2 I_d), \quad p(x) = \mathcal{N}(x; \mu_p, \sigma_p^2 I_d)$$

The *KL*-divergence is given by:

$$D_{KL}(q||p) = \frac{1}{2} \left(\mathcal{K} \left(\frac{\sigma_q^2}{\sigma_p^2} \right) + \frac{||\mu_q - \mu_p||^2}{\sigma_p^2} \right)$$

Distribution of the variances

Distribution of the means

$$\mathcal{K} = \sum_{i=1}^d (\alpha_i - \log \alpha_i - 1)$$

Variational autoencoders (VAE)

We can explicitly set a normal distribution for the latent space:

Encoder $\mathcal{N}_E(x; \mu_\phi, \sigma_\phi^2 I_k)$

Decoder $\mathcal{N}_D(z; \mu_\theta, 1 \cdot I_d)$

Algorithm 7 β -VAE Training Procedure

Require: Dataset of samples $x \sim p_{\text{data}}$, encoder networks $(\mu_\phi(x), \log \sigma_\phi^2(x))$,
decoder network $\mu_\theta(z)$, latent dim k , constants $\beta \geq 0, \sigma^2 > 0$

- 1: **for** each mini-batch $\{x_i\}_{i=1}^B$ **do**
- 2: Encode each x_i : $\mu_i \leftarrow \mu_\phi(x_i), \log \sigma_i^2 \leftarrow \log \sigma_\phi^2(x_i)$
- 3: Sample noise $\epsilon_i \sim \mathcal{N}(0, I_k)$
- 4: Reparameterize: $z_i \leftarrow \mu_i + \sigma_i \odot \epsilon_i$
- 5: Decode mean: $\hat{x}_i \leftarrow \mu_\theta(z_i)$
- 6: Reconstruction loss:

$$\mathcal{L}_{\text{recon}} \leftarrow \frac{1}{B} \sum_{i=1}^B \frac{1}{2\sigma^2} \|x_i - \hat{x}_i\|^2$$

- 7: KL loss to the prior $p_{\text{prior}}(z) = \mathcal{N}(0, I_k)$:

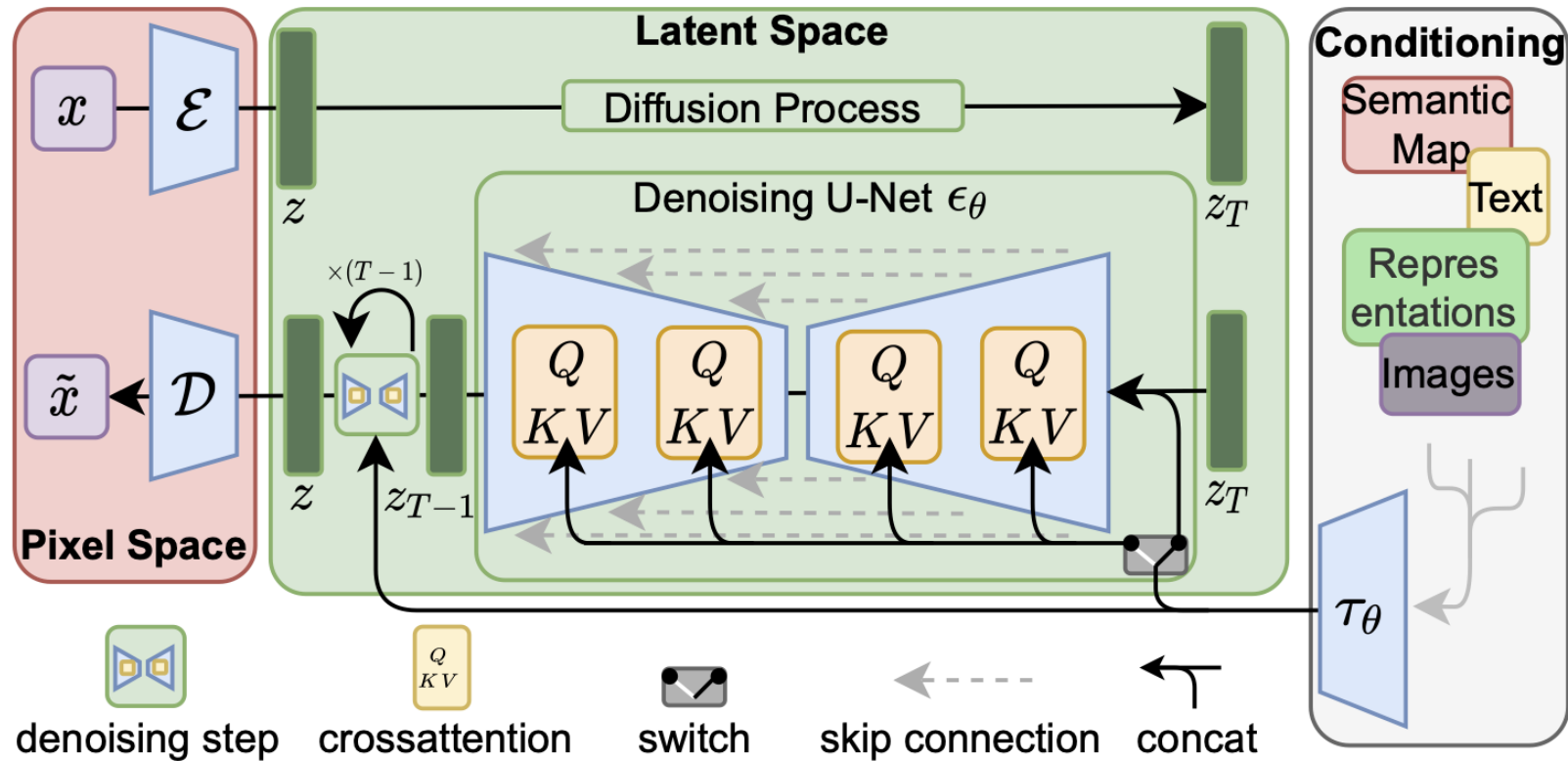
$$\mathcal{L}_{\text{KL}} \leftarrow \frac{1}{B} \sum_{i=1}^B \frac{1}{2} \sum_{j=1}^k (\mu_{i,j}^2 + \sigma_{i,j}^2 - \log \sigma_{i,j}^2 - 1)$$

- 8: Total loss: $\mathcal{L} \leftarrow \mathcal{L}_{\text{recon}} + \beta \mathcal{L}_{\text{KL}}$
 - 9: Update $(\phi, \theta) \leftarrow \text{grad_update}(\mathcal{L})$
 - 10: **end for**
-

Latent diffusion models (LDM)

1. **Data:** take all training data $x_1, \dots, x_N$ (e.g. all images on the internet).
2. **Encode:** convert all images into latents $z_1, \dots, z_N$.
3. **Latent diffusion:** build a diffusion model for the latents – the diffusion model generated latent vectors.
4. **Decode:** sample from the diffusion model and map back to image space.

Latent diffusion models (LDM)



Exact same recipe as before, but with a transformed dataset!

Virtually everything you see is LDM

Stable diffusion: [3, 256, 256] to [4, 32, 32]

FLUX 2.0: [3, 1024, 1024] to [32, 64, 64]



5 | Classifier-free Guidance

Guidance in image generation

Unguided: generate an image.

Guided: generate an image of a cat baking a cake.



A whimsical school bus



A car made of vegetables

Classifier guidance

Algorithm 1 Classifier guided diffusion sampling, given a diffusion model $(\mu_\theta(x_t), \Sigma_\theta(x_t))$, classifier $p_\phi(y|x_t)$, and gradient scale s .

Input: class label y , gradient scale s

$x_T \leftarrow$ sample from $\mathcal{N}(0, \mathbf{I})$

for all t from T to 1 **do**

$\mu, \Sigma \leftarrow \mu_\theta(x_t), \Sigma_\theta(x_t)$

$x_{t-1} \leftarrow$ sample from $\mathcal{N}(\mu + s\Sigma \nabla_{x_t} \log p_\phi(y|x_t), \Sigma)$

end for

return x_0

Classifier-free guidance

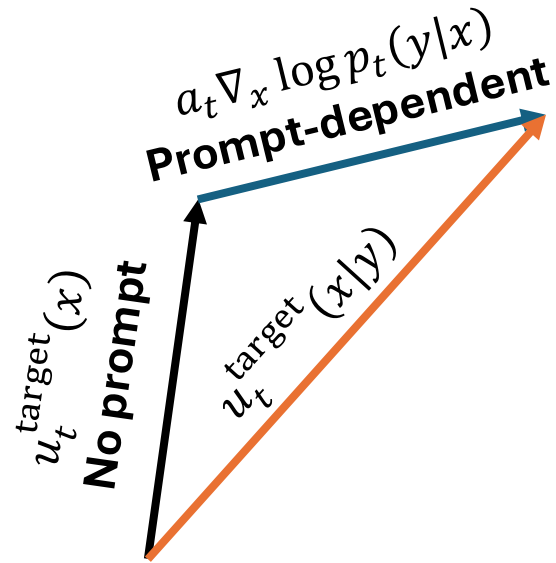
Classifier guidance lead to suboptimal results

Pembroke Welsh corgi

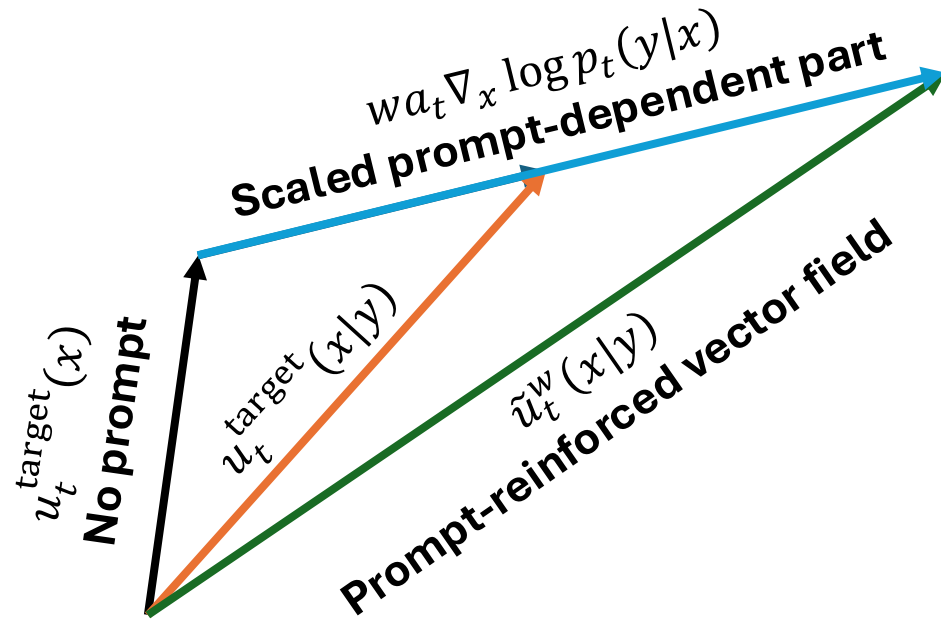


Classifier-Free Diffusion Guidance. Ho and Salimans, 2022.

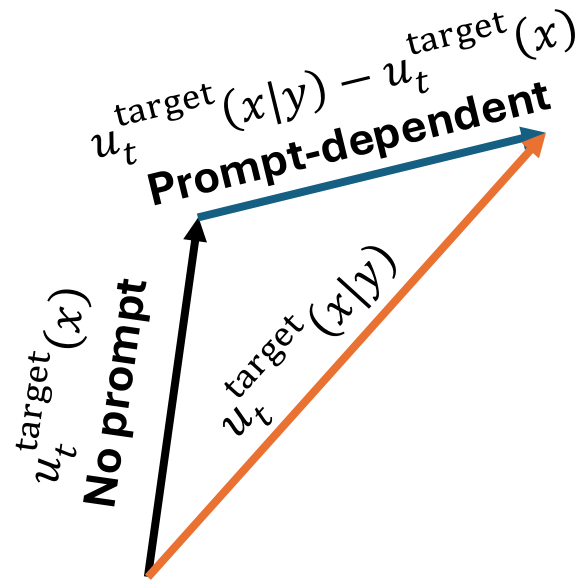
Intuitions: classifier guidance



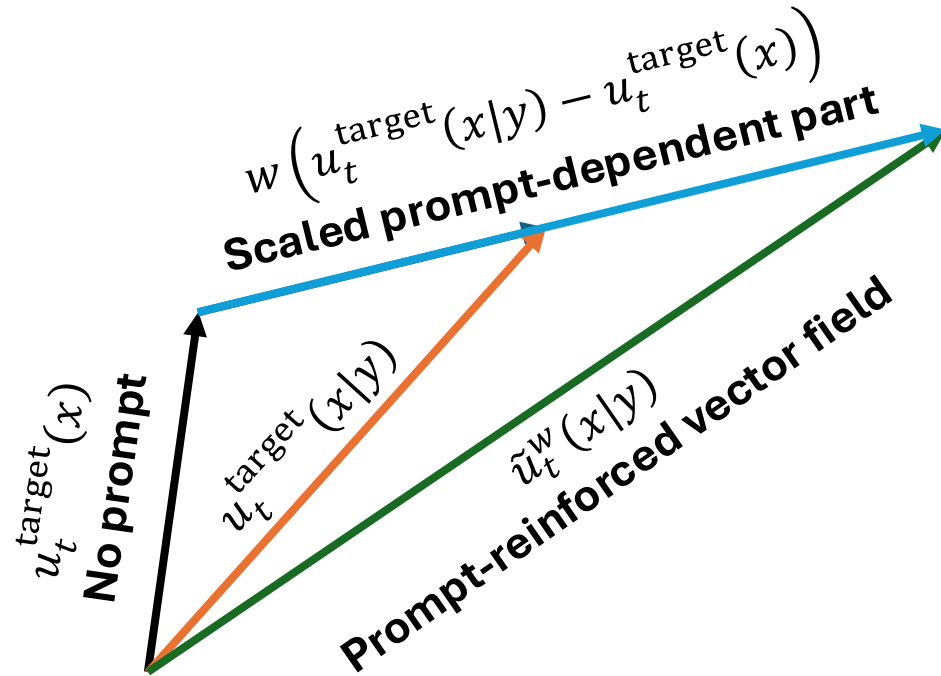
Intuitions: classifier guidance



Intuitions: classifier-free guidance



Intuitions: classifier-free guidance



Classifier-free guidance training

Algorithm 5 Classifier-free guidance training

Require: Paired dataset $(z, y) \sim p_{\text{data}}$, neural network u_t^θ

- 1: **for** each mini-batch of data **do**
- 2: Sample a data example (z, y) from the dataset.
- 3: Sample a random time $t \sim \text{Unif}_{[0,1]}$.
- 4: Sample noise $\epsilon \sim \mathcal{N}(0, I_d)$
- 5: Set $x = \alpha_t z + \beta_t \epsilon$
- 6: With probability p drop label: $y \leftarrow \emptyset$
- 7: Compute loss

$$\mathcal{L}(\theta) = \|u_t^\theta(x|y) - u_t^{\text{target}}(x|z)\|^2$$

- 8: Update the model parameters θ via gradient descent on $\mathcal{L}(\theta)$.
- 9: **end for**

Classifier-free guidance sampling

Algorithm 8 Classifier-Free Guidance Sampling Procedure

Require: A trained guided vector field $u_t^\theta(x|y)$.

- 1: Select a prompt $y \in \mathcal{Y}$, or take $y = \emptyset$ for unguided sampling.
 - 2: Select a **guidance scale** $w > 1$.
 - 3: Initialize $X_0 \sim p_{\text{init}}$.
 - 4: Simulate $dX_t = [(1 - w)u_t^\theta(X_t|\emptyset) + wu_t^\theta(X_t|y)] dt$ from $t = 0$ to $t = 1$.
-

Classifier-free guidance sampling

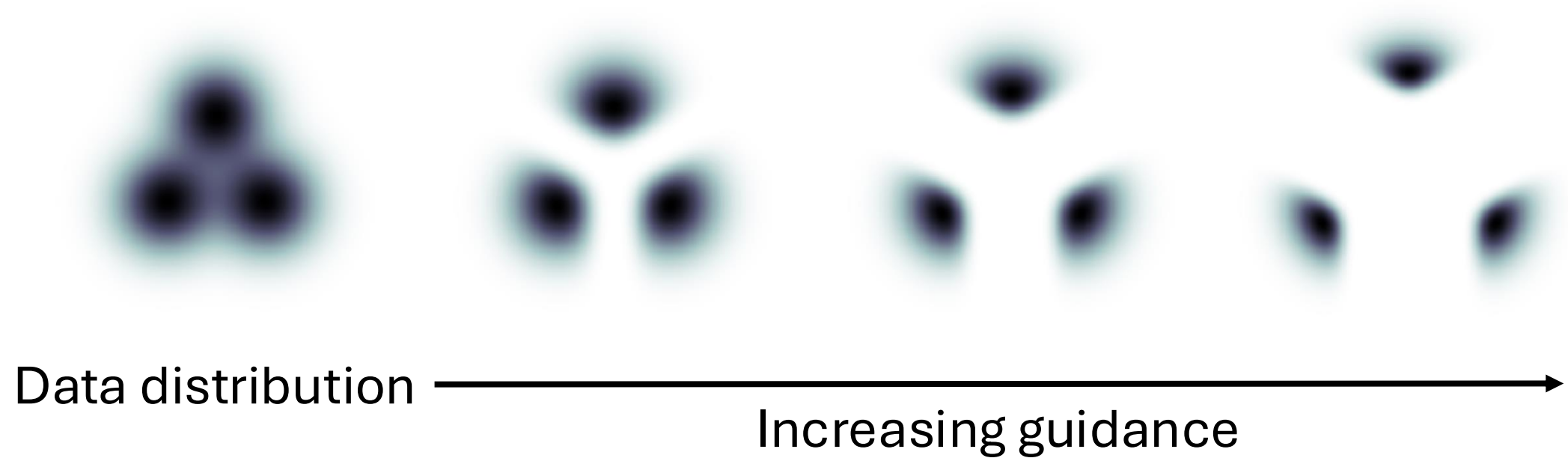


$w = 1$

$w = 4$

* The notations in the original paper are different, there it is $w = 0$ and $w = 3$ respectively.

CFG does not model data distribution



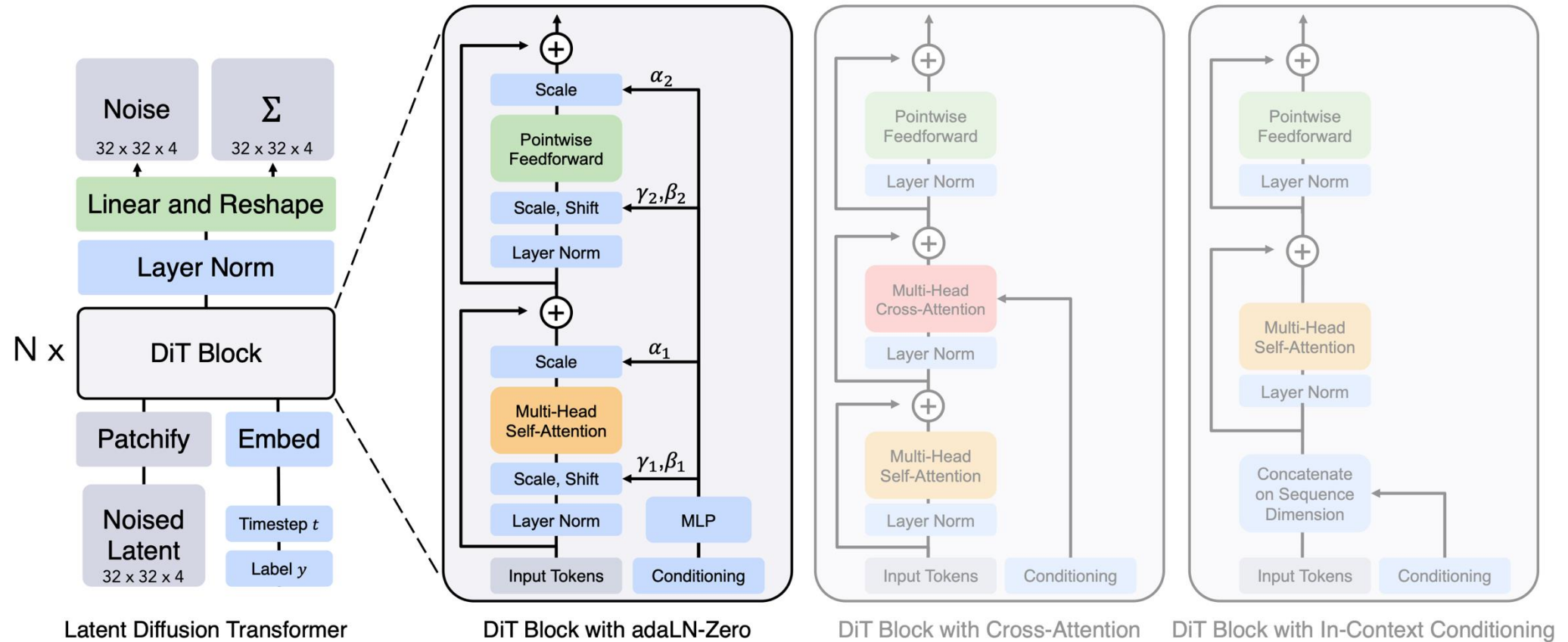
Virtually everything you see is CFG

Stable Diffusion 3: CFG with $w \approx 4.0$



6 | Diffusion Transformers

Diffusion transformer (DiT)



DiT

Use a transformer-based architecture (N ViT blocks) to predict the noise for denoising step t .

The original paper presented 3(+1) conditioning methods:

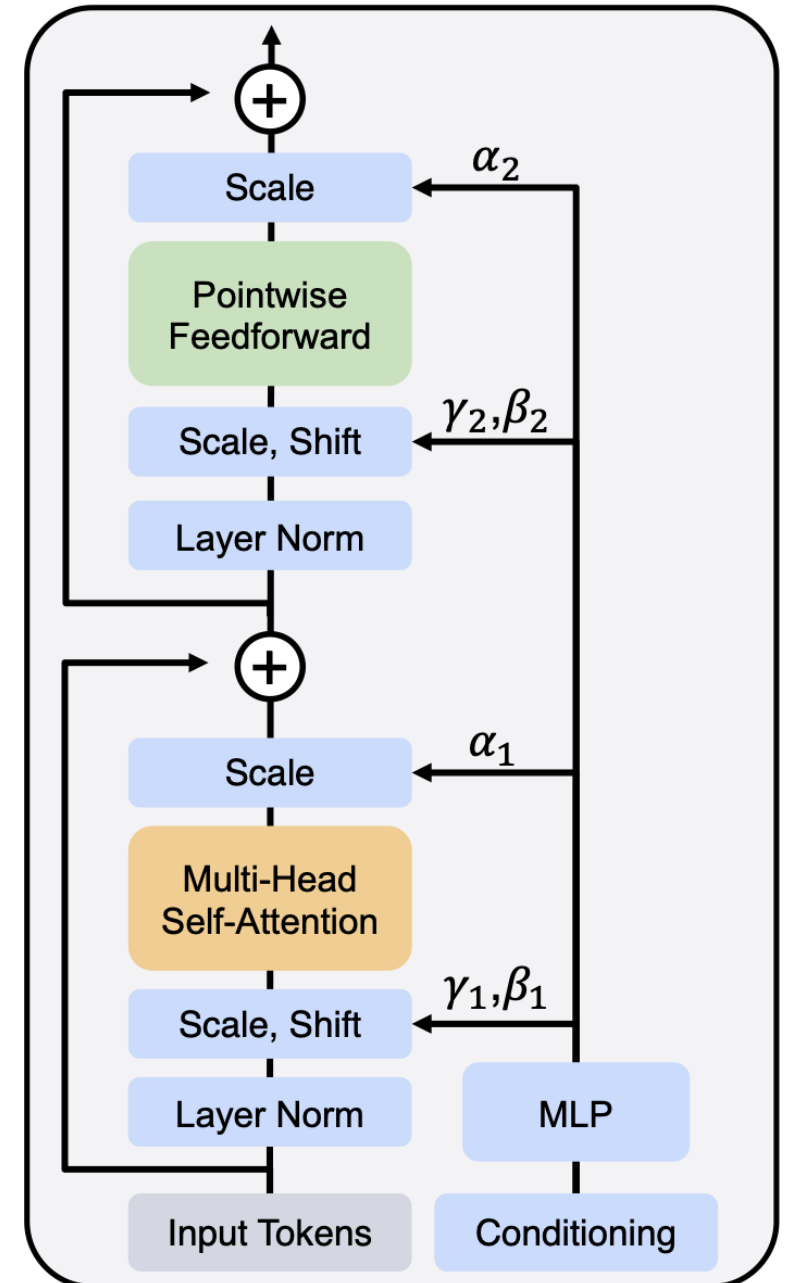
1. **In-context**: append condition embeddings as extra tokens, similarly to *CLS* token in ViT.
 2. **Cross attention**: add an image-to-text cross attention head after the image self attention head.
 3. **Adaptive layer norm (adaLN)**: regress the adaptive layer normalization from the condition embeddings.
- 3+1. adaLN-Zero**: also regress a scaling factor

adaLN and adaLN-Zero

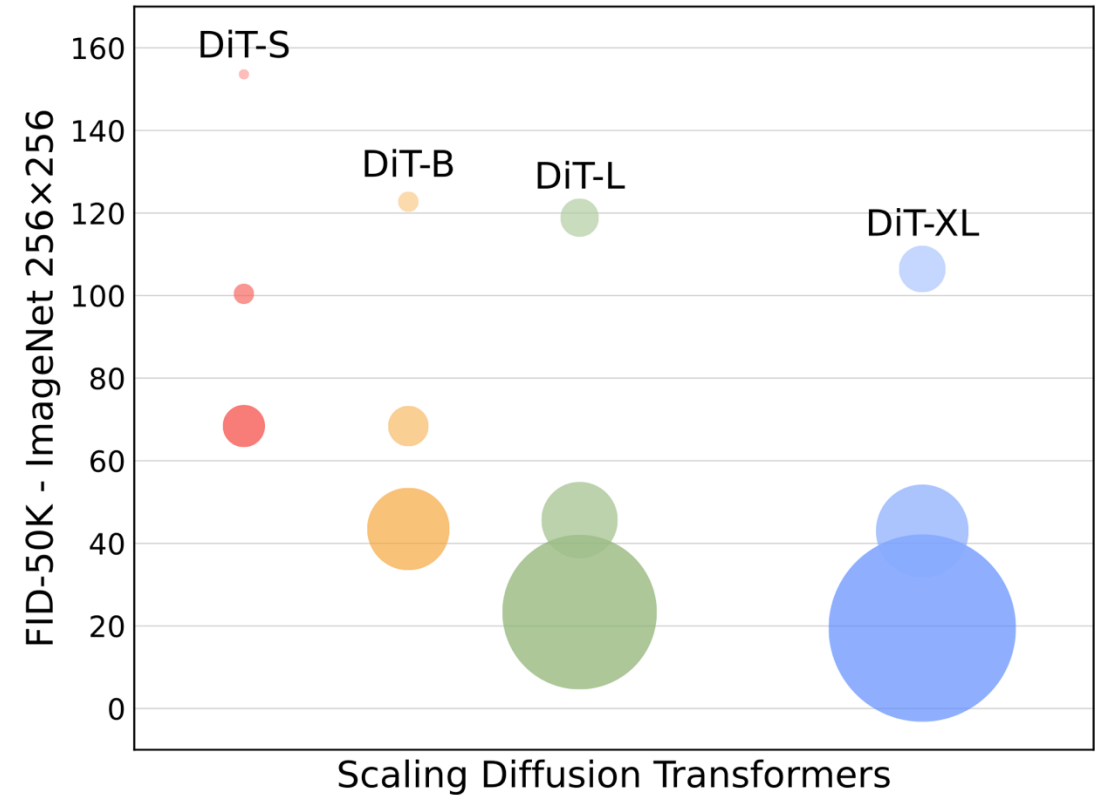
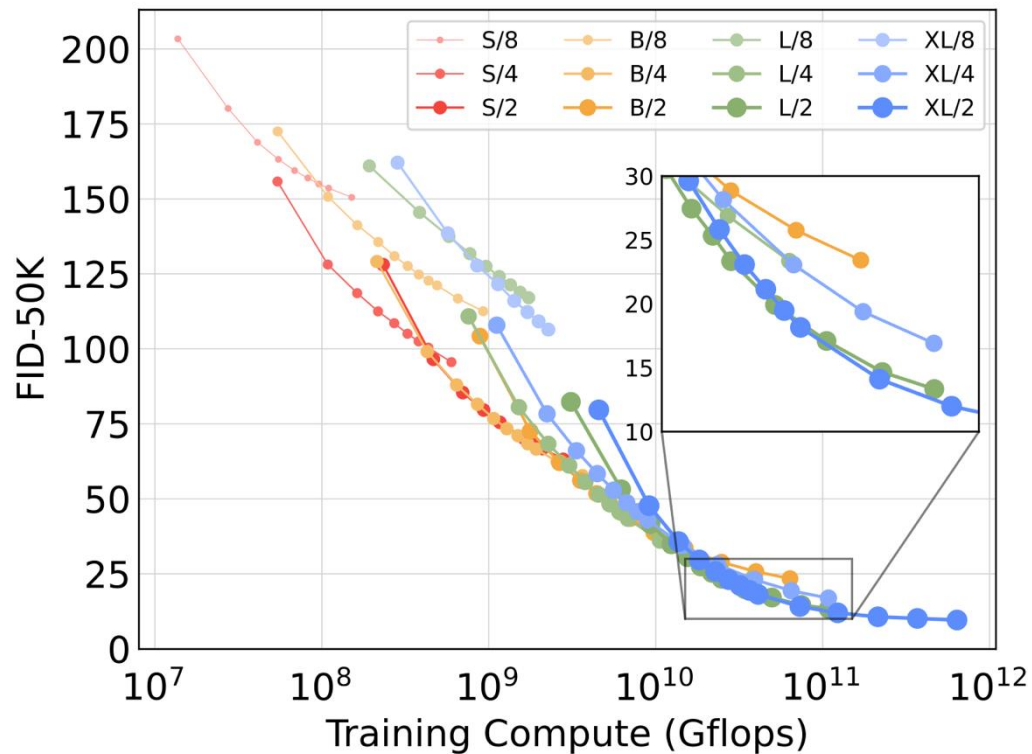
Adaptive normalization was already shown as effective before DiT.

Here, scale γ and shift β are regressed from the condition embeddings.

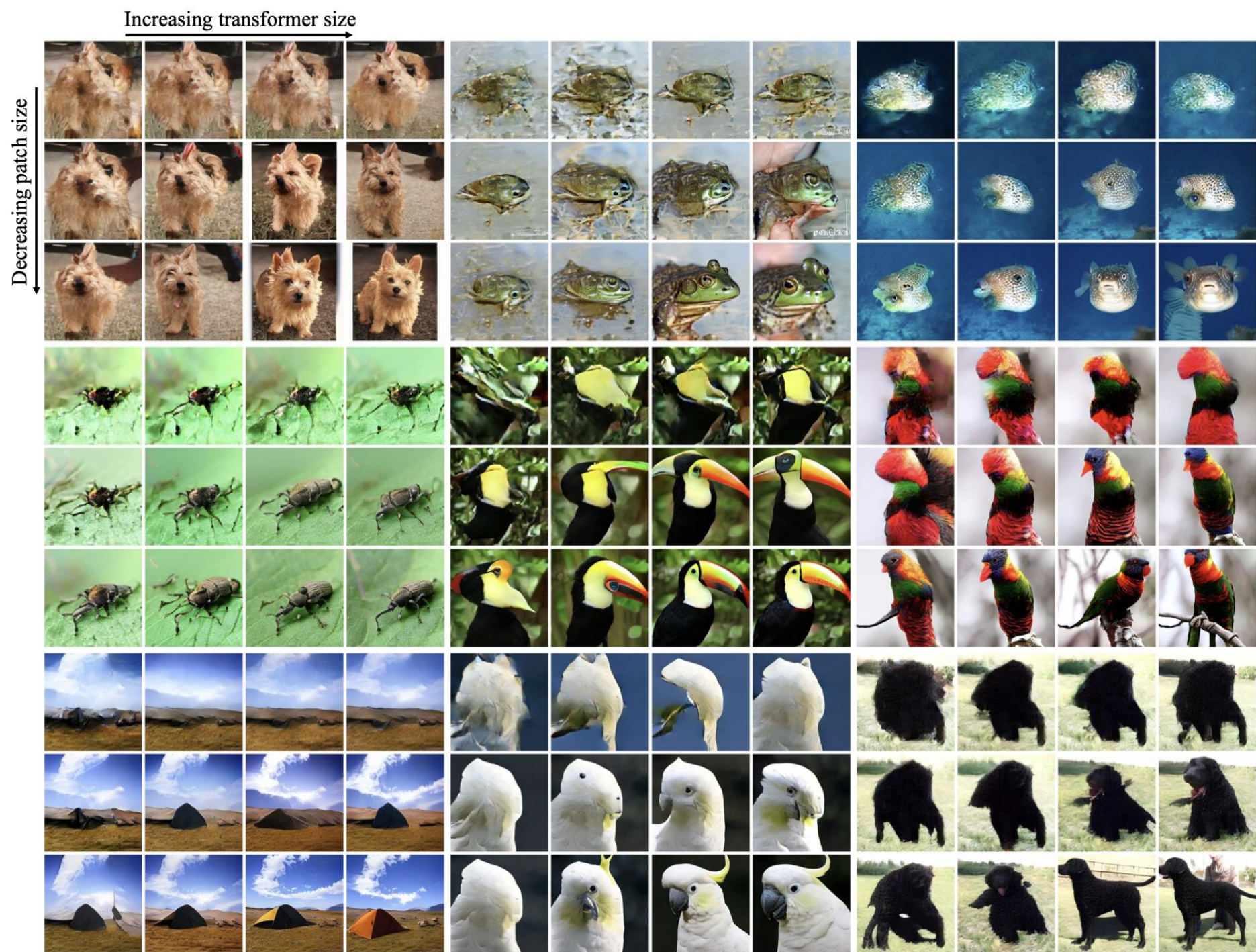
adaLN-Zero adds a conditioned scale factor α and initializes the MLP to output zero-vector for all α s – the DiT is initialized as the identity.



Scaling laws in DiT



Model	Layers N	Hidden size d	Heads	Gflops ($I=32, p=4$)
DiT-S	12	384	6	1.4
DiT-B	12	768	12	5.6
DiT-L	24	1024	16	19.7
DiT-XL	28	1152	16	29.1



Scaling laws in DiT

Scaling performances is straight forward in DiTs:

- Larger transformers perform better
- Larger embedding size perform better
- Smaller patches perform better

Scaling laws in DiT

Scaling performances is straight forward in DiTs:

- Larger transformers perform better
- Larger embedding size perform better
- Smaller patches perform better

More compute means better performances!

Scaling laws in DiT

DiTs (unlike U-Nets) are homogeneous and scale better.

DiTs inherit the scaling behavior previously observed in transformers for NLP and vision.

Extra note: DiTs benefit from the recent advances in transformer optimization.

Multimodal DiT (MM-DiT)

In standard DiT, text is only used to condition the image.

An asymmetry – image tokens change, text tokens remain fixed.

Instead:

Image tokens $\rightarrow$ Transformer $\leftarrow$ Cross attention $\leftarrow$ Text tokens

Multimodal DiT (MM-DiT)

In standard DiT, text is only used to condition the image.

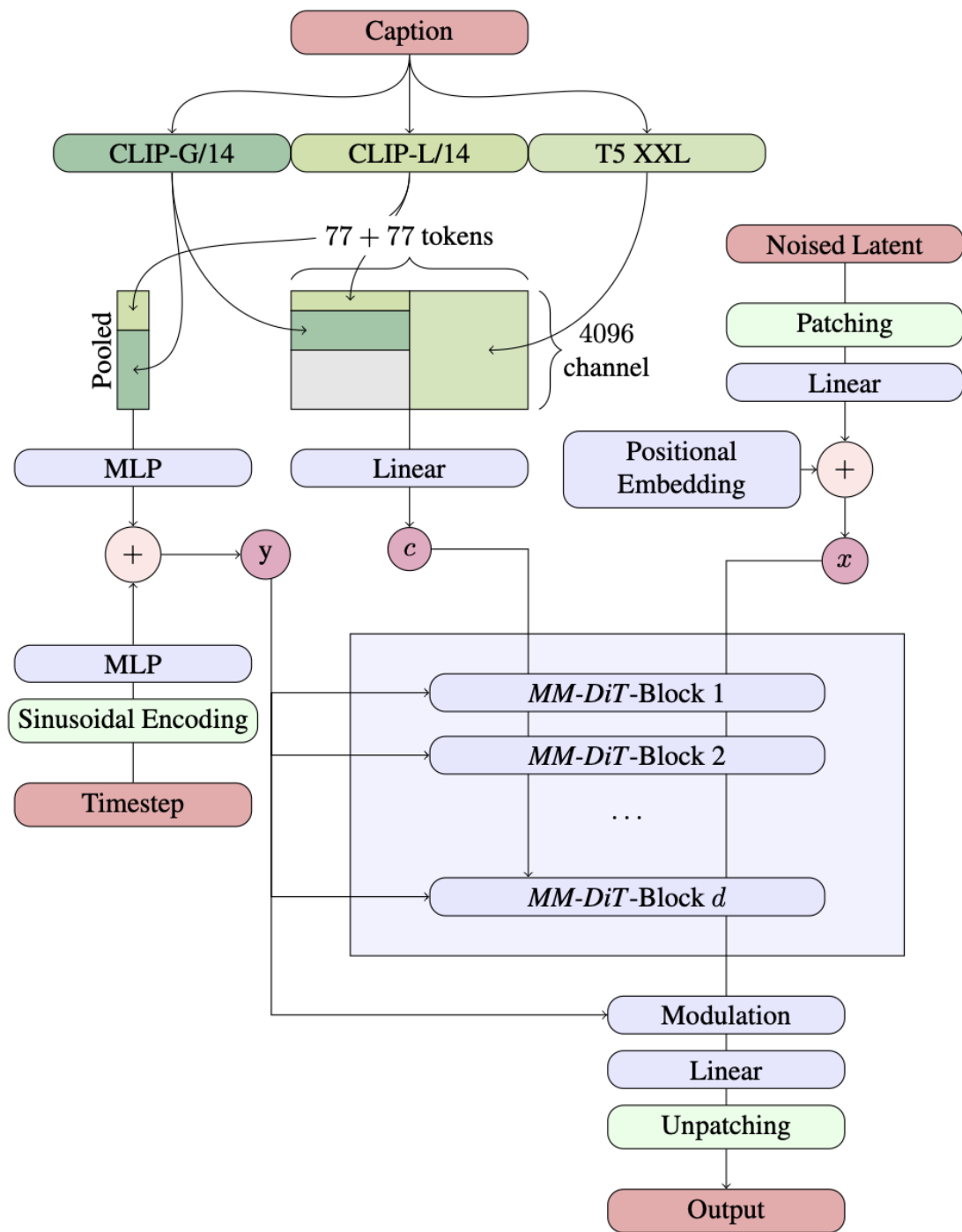
An asymmetry – image tokens change, text tokens remain fixed.

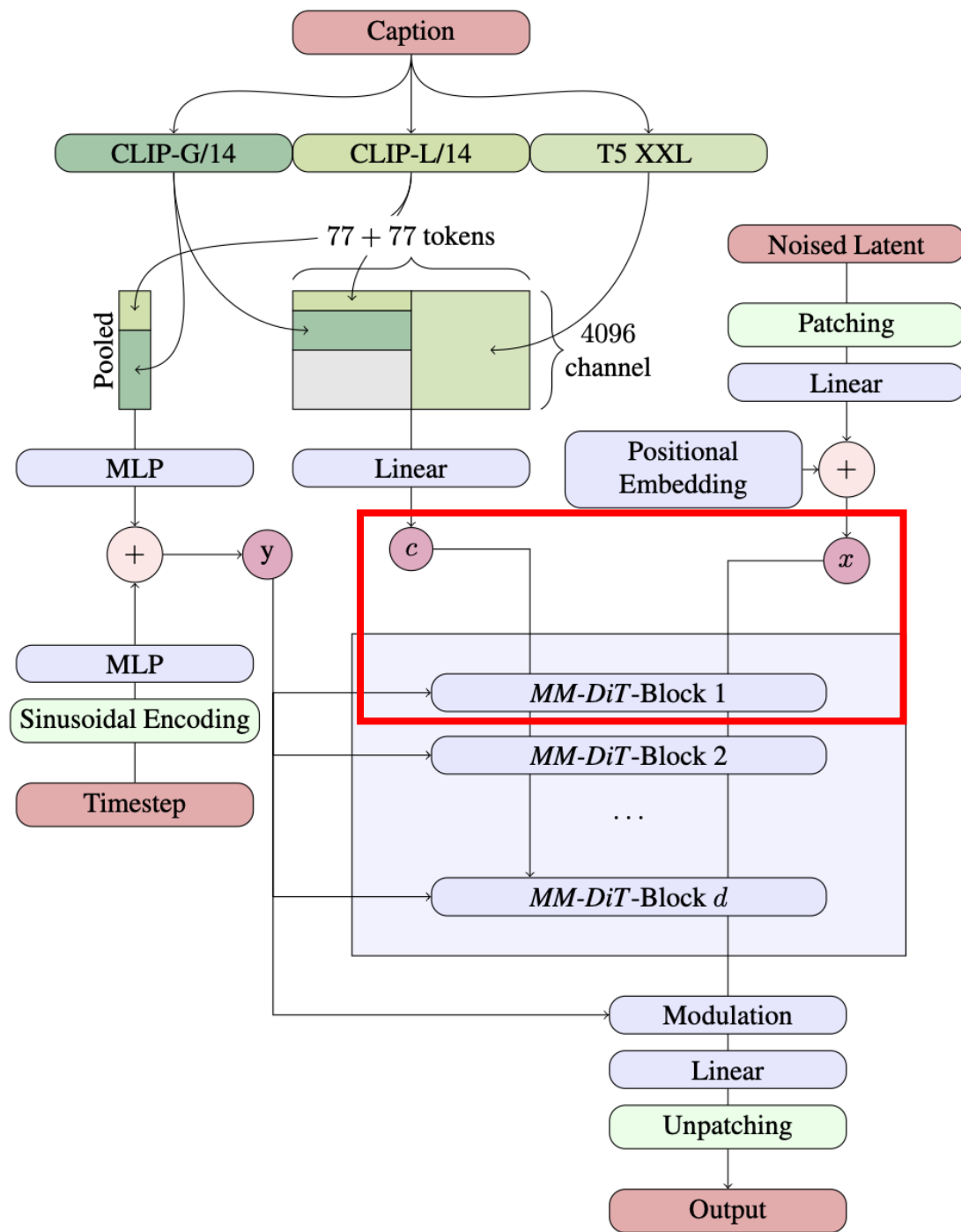
Instead:

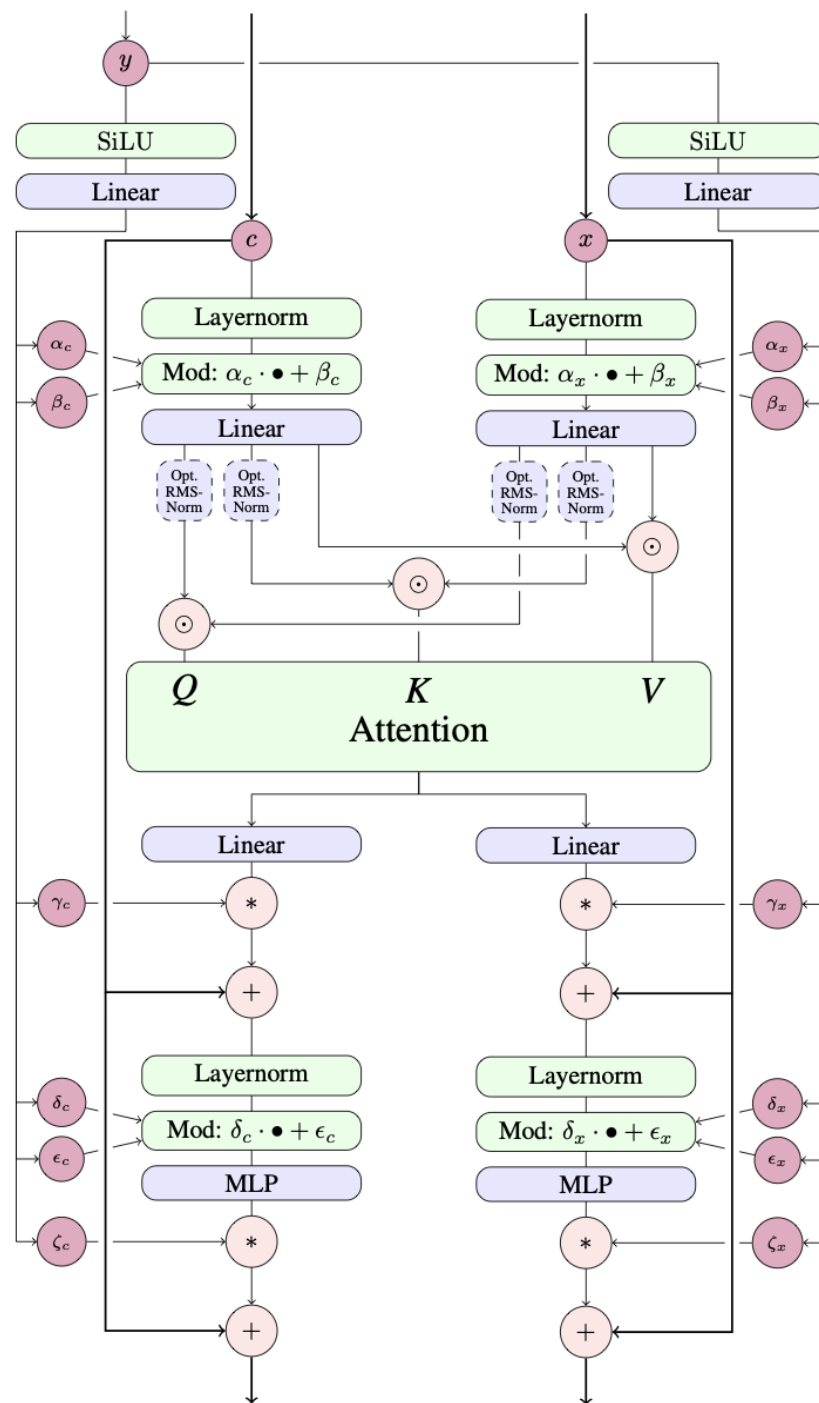
Image tokens $\rightarrow$ Transformer $\leftarrow$ Cross attention $\leftarrow$ Text tokens

MM-DiT use:

Text tokens $\rightarrow$ Joint transformer block $\leftarrow$ Image tokens







MM-DiT

Image and text tokens are concatenated:

$$\begin{aligned} I &= [i_1, \dots, i_N] \\ T &= [t_1, \dots, t_M] \\ [I, T] \end{aligned}$$

But have separate projection matrices:

$$\begin{aligned} W_Q^I, W_K^I, W_V^I \\ W_Q^T, W_K^T, W_V^T \end{aligned}$$

MM-DiT

The attention matrix is then:

	Image	Text
Image	Self attention over the latent image	Standard conditioning signal
Text	Text representation evolves based on the image (New!)	Self attention over text tokens

Only the image tokens are then unpatchified.

Virtually everything you see is DiT

Stable Diffusion 3: MM-DiT



Stable diffusion 3

A general example for this class:

- Multimodal latent diffusion transformer
- CLIP+T5 text encoding
- Classifier-free guidance with $w = 4$